

ECE 3031: Electrical and Computer Engineering Design

Final Design Document

Submission Date:

April 13th, 2026

Adam Bateman 3753751

Alexander Ross 3752371

Corey White 3759566

Ethan Coyle 3768284

Table of Contents

6.2 Project Introduction, Requirements and Preferred Features 3

6.3 Initial Research 5

6.4 Engineering Requirements 9

6.5 Research Results 10

A) H-Bridge Motor Driver Chip 10

B) H-Bridge Motor Driver Evaluation Board 11

C) Discrete H-Bridge Motor Driver 12

D) Analog Controller 13

E) Digital Controller 14

6.6 Final Design Description 14

6.7 Final Detailed Design 17

Final Schematic 17

Controller 18

Multi-Motor Integration 20

Current Measurement 21

Graphical User Interface 24

Motor Driving 26

6.8 Final Component Selection and Bill of Materials 27

6.9 Health, Safety, Environmental, and Social Considerations 28

6.10 User Guide 29

References 31

Appendix A: Test & Verification Plan 32

Appendix B: Gantt Chart 43

Appendix C: Software/Firmware 46

Code for controller: 46

Code for GUI: 63

Table of Figures

Figure 1. Conceptual system overview of the electrical system for the SCORBOT-ER. 4
Figure 2. Block diagram of TI LM62X IC [6]. 6
Figure 3. Torque vs. speed and torque vs. current of SCORBOT motors. 7
Figure 4. Torque vs input power and torque vs. output power of SCORBOT motors. 8
Figure 5. Torque vs. efficiency of SCORBOT motors. 8
Figure 6. Stand-alone H-bridge package from [4]. 11
Figure 7. H-bridge driver integrated on Evaluation board from [7]. 12
Figure 8. Standard H-bridge circuit from [2]. 13
Figure 9. High-level block diagram of control system. 16
Figure 10 - Block diagram of keyboard UI. 17
Figure 11 - Final Schematic 18
Figure 12. Final flow-diagram of motor controller. 20
Figure 13. System model used for Cartesian coordinate conversions. 21
Figure 14. Frequency response of LPF circuit. 22
Figure 15. Time response of 50% duty cycle 4 kHz square wave. 23
Figure 16. Schematic of current measurement circuit. This measures one motor's current, so to measure all motors, so 3 copies of this circuit will be required. 24
Figure 17. GUI for SCORBOT. 25
Figure 18. Second iteration of current measurement design. 43
Figure 19. LTspice schematic of final current measurement design. 43
Figure 20. Project Gantt chart. 44

6.2 Project Introduction, Requirements and Preferred Features

The SCORBOT-ER Vplus is a smaller robotic arm that houses six motors. This is a standard robotic arm that operates in a very similar manner to a level-luffing crane. Three of the on-board motors are responsible for controlling the position of the arm, and three control the gripper that is used to pick up objects [3]. This robot comes with a pre-built controller, but this control system is older and could be replaced with something that uses newer technology. It is for that reason (and as an academic exercise) that our team has been tasked with designing a control system to interface with this arm. We are only responsible for the three motors that control the arm and base rotation.

For this project, our group has been tasked with developing a control system for this type of crane. The following is a list of the current customer (CR) requirements:

CR-1: The SCORBOT-ER should be able to move a negligible mass.

CR-2: The control system must operate the motors within their operating range, to minimize wear on the components.

CR-3: The SCORBOT-ER must be able to draw a straight, diagonal line.

CR-4: The SCORBOT-ER must be able to lift the load over a barrier to an elevated platform.

CR-5: The control system for the SCORBOT-ER must use a MEGA 2560 R3 microcontroller.

CR-6: The control system must operate at a safe speed and torque to avoid possibility of injury.

CR-7: The implementation of the control system should be done in a cost-effective manner.

A conceptual system overview of the electrical system for the SCORBOT-ER is shown below in Figure 1.

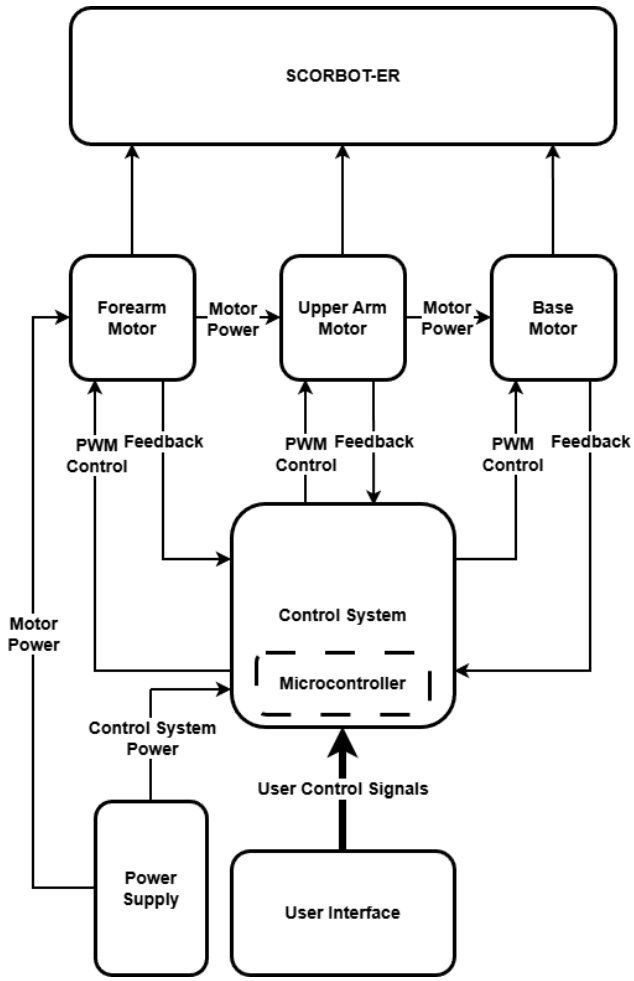


Figure 1. Conceptual system overview of the electrical system for the SCORBOT-ER.

User Interface: The user interface will translate commands coming from the real-world into digital commands that are sent to the control system to operate of the SCORBOT-ER. This sub-component will likely require little amounts of work with an initial estimate being approximately 5 hours to develop.

Control System: The control system is the main sub-component of our design. Its purpose is to control the forearm motor, upper arm motor, and base motor by sending control signals to adjust their speeds and torque, while monitoring feedback information on their relative positions and speeds. The control system will interpret and respond to commands from the User Interface. Currently, the user interface consists of a MEGA 2560 R3 microcontroller. This sub-component will likely require substantial amounts of work with an initial estimate being approximately 20 hours for reliable operation. This sub-component presents no safety risks because of the low voltages it uses to operate.

Base Motor: The base motor is responsible for controlling rotation of the SCORBOT-ER. It will receive input and provide feedback to control position and velocity. All motors and actuators are included in the SCORBOT-ER provided by the client, so they will require minimal work to implement. In terms of safety, the base motor is relatively low risk since it is not located in an area highly accessible by the user.

Forearm Motor: The forearm motor is responsible for controlling the position and speed of the load. It will interact with the Control System by receiving input and providing feedback to control position and velocity. This is a relatively low risk component

Upper Arm Motor: The upper arm motor will control the position and speed of the load. It will be receiving input and provide feedback to control position and velocity. This upper arm motor will likely be low risk and will not be capable of causing any harm to individuals around it.

Power Supply: The power supply will be used to power the motors and the control system. Since information on the motors has not been provided, selection of a power supply at this stage cannot be made. This sub-component will likely require little amounts of work to adequately select with an initial estimate being approximately 3 hours. In terms of safety, this power supply will be low risk since it will likely be low voltage.

SCORBOT-ER: The SCORBOT-ER is the main sub-component that will be developed by the professor. Initial estimates of work cannot be made since this task is not performed by this group. In terms of safety, this SCORBOT-ER is low risk since it is relatively small and does not handle large masses; however, safety precautions should be taken around motors and actuators.

6.3 Initial Research

Before we designed anything specific, we researched what others had done in the process of designing similar control systems. This helped us determine how feasible solutions were, and whether or not we should go ahead with them. We already knew that our embedded system of choice was the Elegoo board, but how we interface with it was still unknown. Through this research process, we developed a list of potential solutions for our project.

The main challenge of this project was controlling the three motors on the SCORBOT for which we found several possible solutions. We wanted to be able to reverse the direction of our motors. A popular solution to this problem is using an H-bridge, [2] a motor driver made from transistors, often packaged in an IC. We could have also tried to build our own motor driver, but there are several challenges with this, as stated in [2]. These challenges include but are not limited to inductive kick-back from the motor windings damaging the transistors, and difficulties timing the switching of the transistors, to not short anything out. Therefore, an easier solution was to use a pre-built H-bridge, as stated in [2]. On top of this, our professor recommended that we use pre-built H-Bridges, as to not ‘re-invent the wheel’.

Having looked at the broader market, we could see that larger manufacturers use more software to interface with motors on their higher-end products. However, they still use H-bridges to interface with the motor. If we look at the datasheet for the Texas Instruments LM629M-8 [6], we can see that their product has a desired position input, a digital PID controller, a DAC and an op-amp. Their design also uses negative feedback, but that is standard in most applications. A diagram for this can be seen in Figure 2.

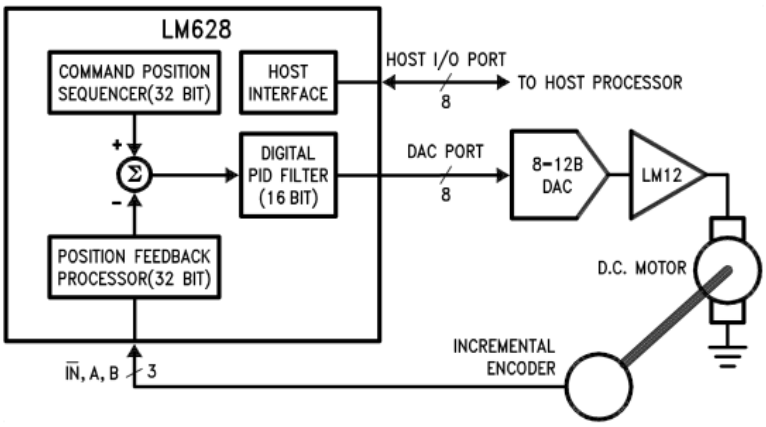


Figure 2. Block diagram of TI LM62X IC [6].

Another solution was Performance Motion Device's MC54113. It is also complicated, being able to support both DC and Step motors, SPI output, and a bridge amplifier to directly drive a motor [5]. Furthermore, our professor recommended the use of H-bridges on pre-made evaluation boards that include things like protection circuitry and having pins for soldering terminals as well.

The goal of this project is to design a controller, which means that we should not have bought one of the pre-made ICs, but we could still have taken inspiration from them, thus, we had derived the following list of potential solutions:

- H-Bridge Motor Driver Chips
- H-Bridge Motor Driver Evaluation Board
- Discrete H-Bridge Driver (Transistor Design)

Each of these solutions could have been implemented using digital signal processing or analog processing. These methods have their own advantages and disadvantages, such as the analog processing being faster but more difficult to implement. It is worth noting that motor controllers such as the LM269M-8 work mostly with digital signals and digital controllers, so they have some advantages over analog controllers.

The motors on the SCORBOT-ER are Pittman brushed DC motors. Motors 1, 2, and 3 (the motors we must interface with) are all the GM9413G608 model. Before calculating anything, we had put the motor's operating parameters into a MATLAB file made by Pololu to get the motor characteristic graphs of the motor being used.

As specified by the professor, the current through the motors should not go above 2 A. We had modified the MATLAB script that was found on D2L to generate Figure 3, Figure 4, and Figure 5 This means that the maximum torque that is achievable with this is about 6 oz-in as shown in Figure 3. The first plot represents the torque versus speed characteristic of the DC motor. The second plot represents the torque versus output power and torque versus input power characteristic of the DC motor. The third plot represents the torque versus power efficiency relationship for the DC motor. This can be used to calculate the maximum mass of the arm by modelling the arm as a rod where τ is torque, α is angular acceleration, and M and l are the mass and length of the arm, respectively. Of course, this model will not give the exact limit, but it should give a ballpark of what the motor can handle.

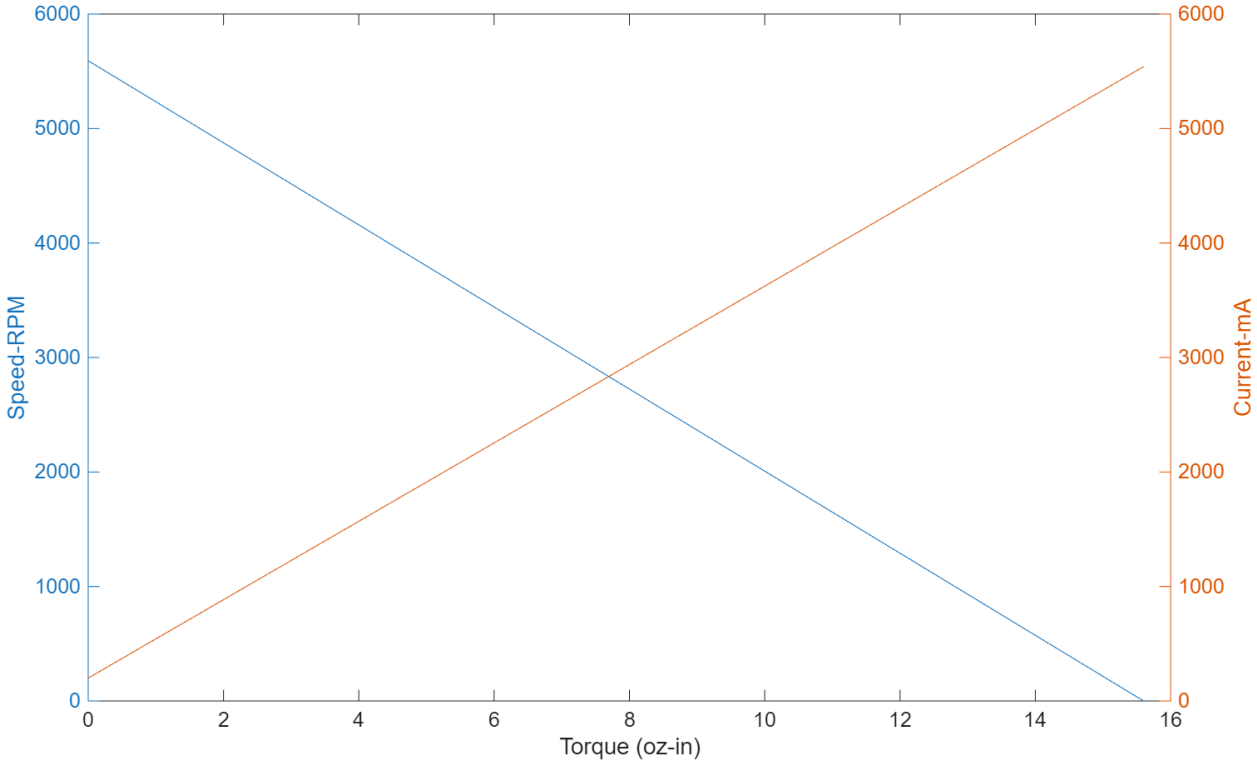


Figure 3. Torque vs. speed and torque vs. current of SCORBOT motors.

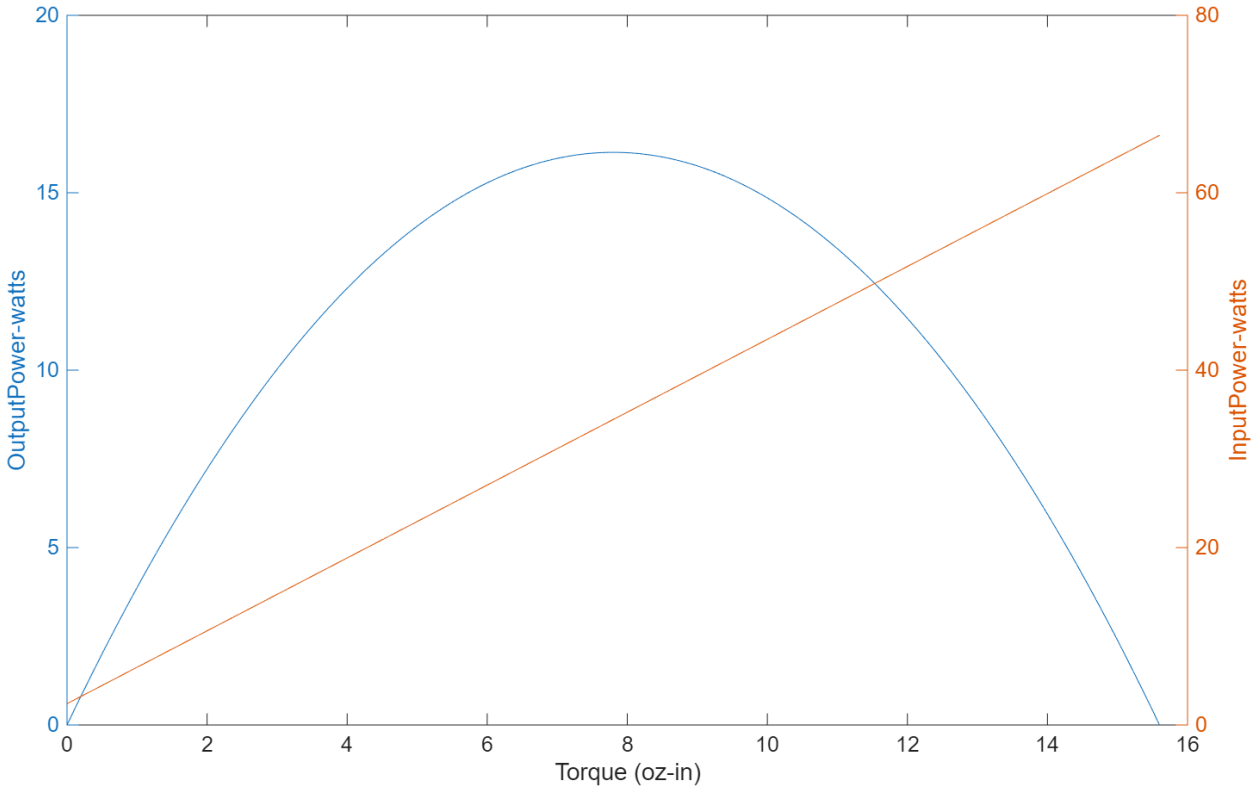


Figure 4. Torque vs input power and torque vs. output power of SCORBOT motors.

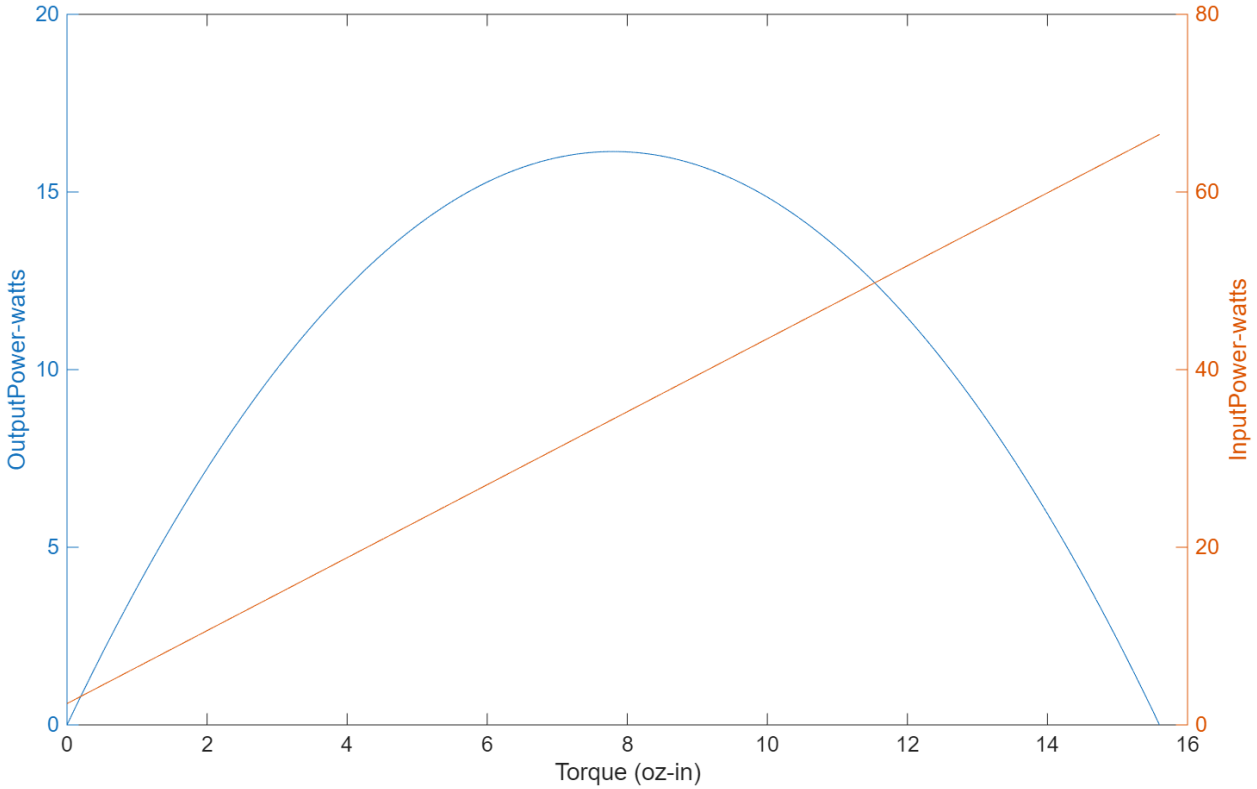


Figure 5. Torque vs. efficiency of SCORBOT motors.

$$\tau = \frac{1}{3} M l^2 * \alpha \quad (1)$$

We had chosen the value of our desired angular acceleration, was 0.25 rad/s^2, while acknowledging this value was subject to change. Completing further calculations, we determined each of the arm segments is 220 mm long [3], but the gripper is also included, which adds about 100 mm. Thus, the length of the extended arm should be no larger than 540 mm. The torque can also be converted into newton-meters. The maximum speed of the arm is 23.6 degrees/second [3], and we want to get to that speed in 1 second, so we will set our angular acceleration to 23.6 degrees/second^2. We can use this information to find the required output torque, τ_{out}, in Equation (2) and Equation (3) and use the gear ratio to find the output torque, τ_{in} in Equation (5). These values can be converted to oz-inches, seen in Equation (4) and Equation (6).

$$\tau_{out} = \frac{1}{3}(1.5)(0.220 + 0.220 + 0.100)^2 * (23.6 * \frac{\pi}{180}) \quad (2)$$

$$\tau_{out} = 0.074142\ Nm \quad (3)$$

$$\tau_{out} = \frac{0.074142\ Nm}{0.00706\ \frac{Nm}{oz * in}} = 10.50\ oz * in \quad (4)$$

$$\tau_{in} = \frac{\tau_{out}}{127.7} = 5.8059 \times 10^{-4}\ Nm \quad (5)$$

$$\tau_{in} = \frac{5.8059 \times 10^{-4}\ Nm}{0.00706\ \frac{Nm}{oz * in}} = 0.0822368\ oz * in \quad (6)$$

The gearing ratio, as was specified by [3], is 127.7:1. This has the effect of reducing the required current for the motor. Looking at the torque-current characteristic graph, with the gearing included, we were able to achieve our desired torque by supplying less than 500 mA, which is well within the 2A requirement. We only calculated the torque for the motor that had to drive the base of the arm, as that motor would have to support the maximum load.

6.4 Engineering Requirements

The following is a list of Engineering Requirements (ER) that expand on the Customer Requirements (CR) listed above:

ER-1: The SCORBOT-ER must be able to maintain structural integrity and provide consistent motor torque capable of moving the robot.

ER-2: The SCORBOT-ER control system must be able to autonomously draw a diagonal line at a 45° angle while not deviating more than 10 mm over its intended path, for a distance of at least 500 mm by using a closed-loop feedback system to monitor the position and velocity of the load.

ER-3: The SCORBOT-ER' gripper must be capable of traversing over a 27 cm barrier and returning to a platform with a height of 13 cm while travelling a total horizontal distance of 25 cm.

ER-4: The control system must be capable of simultaneously controlling the forearm motor, the upper arm motor, and the base motor~~,~~ to control the position, velocity, and acceleration of the SCORBOT-ER.

ER-5: A suitable power supply must be selected to power the control system and the motors; however, a power supply cannot be selected until more is known about the motors.

ER-6: All the motors used must be capable of being controlled by a PWM source to allow for variable motor speed.

ER-7: The control system for the ~~crane~~ SCORBOT-ER must use the Elegoo MEGA board.

ER-8: No motor shall ever exceed an angular speed of 120 degrees per second regardless of mode of operation.

ER-9: The overall cost of the control system, including the motor drivers and current limiting resistors, must not exceed a total cost of $50.

6.5 Research Results

The three possible solutions for motor control that we decided were plausible would be to make our own H-bridges, buy premade H-bridge motor drivers or to purchase H-bridge motor drivers which are incorporated on evaluation boards. The pros and cons of a premade H-bridge design are listed in Table 1. The pros and cons of purchasing a full motor driver integrated on an evaluation board are shown in Table 2. The pros and cons of making our own motor driver can are shown in Table 3. Additionally, we also had to decide whether we wanted our controller to be an analog or digital, for which, the comparisons are shown in Table 4 and Table 5, respectively. Seen in (7) is the equation we used for estimating the total cost of each solution. Each solution will take different amounts of time to implement; thus, we decided that our work rate will be $15.90. Additionally, we must consider the hardware cost for each solution to determine whether it's feasible.

$$Cost\ Estimate = Rate * Time + Hardware\ Cost \tag{7}$$

A) H-Bridge Motor Driver Chip

By choosing to use a stand-alone H-bridge, for which an example can be seen in Figure 6, we would have to buy one driver per motor along with any additional components for protection on both ends. This design would cost slightly more than creating our own H-bridge, however, the amount of time we would save by not having to design our own H-bridge far outweighs the higher

price. A total cost estimation including implementation time and money spent on protection circuitry can be found in Equation (8). On average, on Digikey, the price of a motor driver and relevant protection circuitry that meets our needs is approximately $5 and we expect that we will need to put a total of 4 hours of work into the design. The pros and cons of a premade H-bridge design are listed in Table 1.

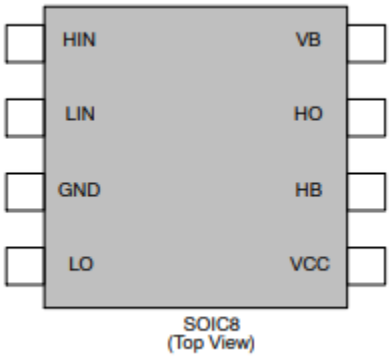


Figure 6. Stand-alone H-bridge package from [4].

Table 1 - Pros and cons of standalone H-bridge motor driver chip for motor control.

Pros	Cons
- Allows for custom protection to best fit our needs. - Low cost per unit. - Easy to wire to board, only three inputs per motor. - Supports PWM, which allows for proportional control of motors. - Easy directional control using two pins.	- Limited to through-hole chips. - Requires additional components (i.e., capacitors) to aid functionality. - More expensive than a discrete design. - Less functionality than an evaluation board.

$$\$15.90 * 4 + \$5 * 3 = \$78.60 \quad (8)$$

B) H-Bridge Motor Driver Evaluation Board

This device includes the H-bridge with more electronic components to add protection circuitry; an image of the device can be seen in Figure 7. These are often more expensive but have a considerable portion of the implementation work done already. A total cost estimation for the motor driver integrated onto an evaluation board can be seen in Equation (9) with the time taken to find and assemble very low. The price of motor driver evaluation boards on Digikey were found

to be approximately $10 and we expect that only 1 hour of work would be required to implement the design. The pros and cons of purchasing a motor driver integrated into an evaluation board are shown in Table 2.

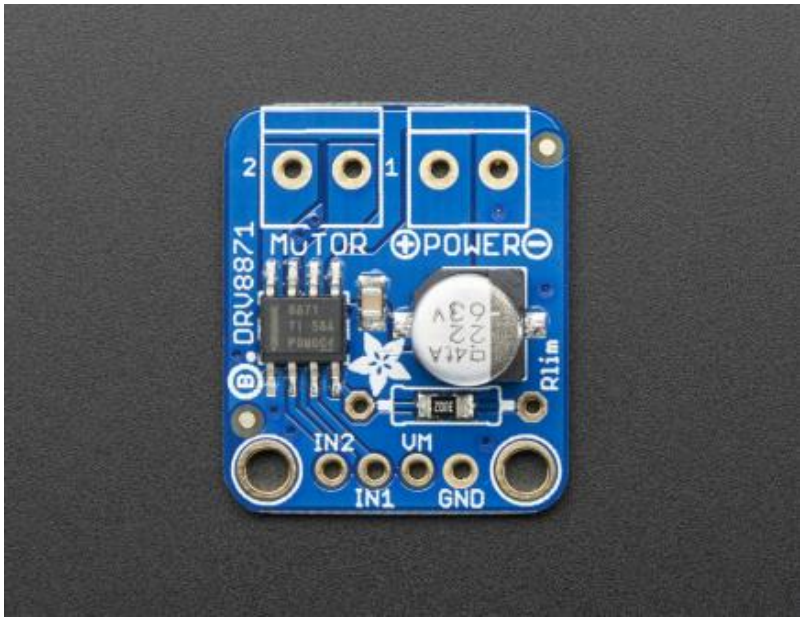


Figure 7. H-bridge driver integrated on Evaluation board from [7].

Table 2 - Pros and cons of H-bridge motor driver evaluation board for motor control.

Pros	Cons
- Already integrated into an evaluation board and functionality is verified. - Allows us to use surface-mount-components which are often manufactured more. - Many surface-mount-chips are more capable than the available through-hole chips. - Has pins for soldering wires, pin headers, terminal blocks, etc., to.	- Higher cost due to the additional components and manufacturing premiums. - Extra functionality which is unnecessary.

$$\$15.90 * 1 + \$10 * 3 = \$45.90 \quad (9)$$

C) Discrete H-Bridge Motor Driver

If we chose the discrete H-Bridge driver, then we would develop our own motor driver using MOSFETs, similar to the circuit seen in Figure 8. We would use the smaller voltage coming from the Elegoo board to control a larger voltage, coming from the power supply. We would have

to implement protection circuitry to mitigate things like voltage spikes and shoot-through as well. The overall cost of the required materials would likely be quite minimal, although the time required to select components, assemble the custom H-bridge, and verify its functionality is much more than many of the other solutions, resulting in the final cost estimation seen in Equation (10). The cost of each H-bridge was estimated using components on Digikey to be $4, noting that each transistor typically has an accompanying Schottky diode, and we expect that we will need to put approximately 8 hours into implementing the design. Additionally, the pros and cons of making our own motor driver can are shown in Table 3.

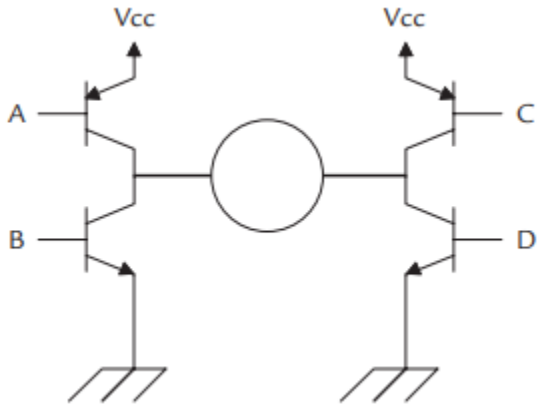


Figure 8. Standard H-bridge circuit from [2].

Table 3 - Pros and cons of discrete H-bridge for motor control.

Pros	Cons
- Highly configurable in terms of motor voltage selection and maximum current. - Provides us with H-bridge design experience. - Lower material cost.	- Time-consuming to implement and verify functionality. - Requires additional circuitry for protection.

$$\$15.90 * 8 + \$4 * 3 = \$133.2 \quad (10)$$

D) Analog Controller

A solution for a controller is an analog controller. An analog controller would still interface with the Elegoo board, but the main portion of the input and output signal processing would be done using analog electronics. For example, the PID controller would be implemented using op-amps in different configurations. This implementation comes with a lot of design considerations

that must be made. Additionally, verification of this design would take a large amount of time to ensure correct functionality. A list of the pros and cons of an analog controller are shown in Table 4.

Table 4 - Pros and cons of an analog controller for the motors.

Pros	Cons
- Very responsive to changes in the system. - Provides us with valuable design experience.	- Not easily configurable (i.e., requires physical modification of components). - If implementing a controller with integrator action, integrator windup is more difficult to prevent. - Power supplies

E) Digital Controller

Another solution for a controller is a digital controller. If a digital controller is used, then all of the signal processing would be done using the Elegoo board. The design will still require external components, such as the desired motor driver, but things like the PID controller will be developed in software. This design would take much less time to implement when compared to the analog controller.

Table 5 - Pros and cons of a digital controller for the motors.

Pros	Cons
- Can easily configure parameters to control desired response. - Easy to prevent integrator windup with code.	- Limited sampling rate can affect the response of the system.

Assuming all potential designs are implemented correctly, they should all meet the engineering requirements. All solutions that we have listed will give us sufficient control over the motors and H-Bridge motor drivers seem to be an elegant solution, as they are simple, easy to control, and relatively economical. We found a motor driver model on Digikey that looks good for our application, which features directional control and PWM inputs.

6.6 Final Design Description

The design we implemented was based around a microcontroller, three motor drivers, and three motors. The microcontroller was used to implement the control algorithm for controlling all three of the motors at the same time. The control algorithm that we developed was a proportional-integral (PI) controller. This controller was used to autonomously navigate the SCORBOT about a planned path. Using the microcontroller, the encoders of the motors are continuously monitored

so the position, velocity, and acceleration of the upper-arm, forearm, and base of the SCORBOT can be tracked. A high-level block diagram of our system is shown below in Figure 9. The following is a list of descriptions of the components that can be found in the high-level block diagram shown in Figure 9:

Microcontroller: The microcontroller was the main component of our project. It was used to implement the control algorithm for controlling the base motor, upper arm, and forearm motor of the SCORBOT.

PIDs: There were three PIDs shown in Figure 9: the base PID, the upper arm PID, and the forearm PID. These PID blocks are used to represent the PID controllers that will be implemented within the microcontroller to control their respective motors. Individual PID control of each motor allowed for the specific control of each joint permitting multi-axis motion while reducing errors and maintaining stability.

PWM Modules: Our system consists of six Pulse-Width-Modulation (PWM) pins, two for each of the motor drivers. These PWM pins used the control value output from the PID controllers to adjust the PWM duty cycle.

Motor Drivers: Our system consisted of three motor drivers, one for each of the motors shown in Figure 9. These motor drivers, as the name suggests, are used to drive their respective motors with the appropriate current and voltage.

Encoder Processors: The encoder processor blocks were used to process the encoder signals sent from the motors to determine the velocity of each of the arms of the SCORBOT.

Motor Position, Speed, and Acceleration Processor: This block was the block in our control system that processed the data from the encoder processors to determine parameters such as position, speed, and acceleration. Based on these measurements, the controller output a new desired velocity to maintain the SCORBOT's intended path. This type of closed-loop control allows for accurate tracking and adjustments especially in autonomous modes while handling object traversal.

Computer: A computer is our main method of interfacing with the controller. The controller output data to the terminal of the computer to assist in system monitoring and allow for user interference and debugging when necessary. Additionally, through the computer and keyboard, the user will be able to send commands to the controller to manually control the SCORBOT. A block diagram of the user interface of the keyboard interfaced to the computer can be seen in Figure 10. The keys shown will allow for mode selection, enabling/disabling of the SCORBOT while in either autonomous mode or manual mode, and arrows to control the robot manually when desired.

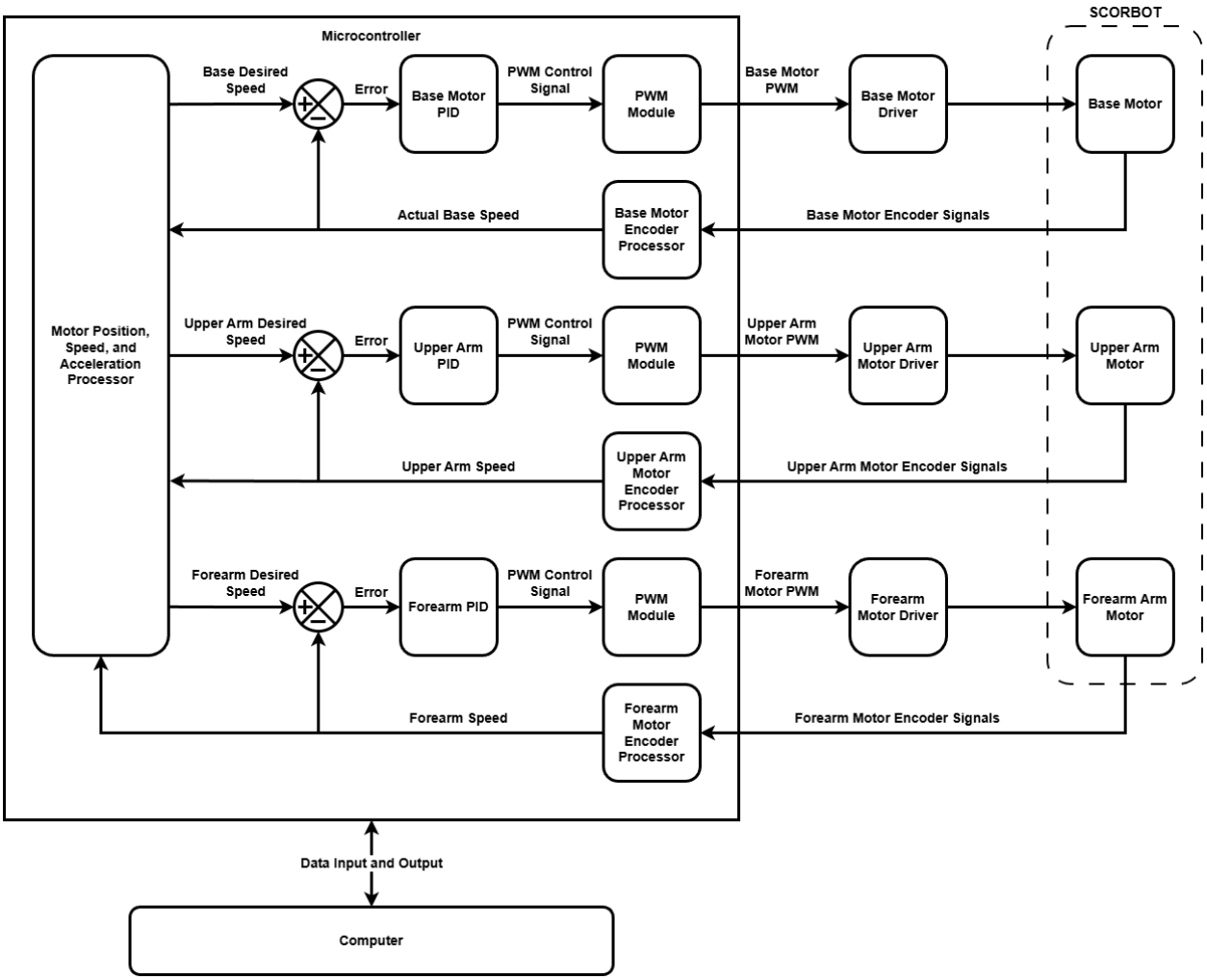


Figure 9. High-level block diagram of control system.

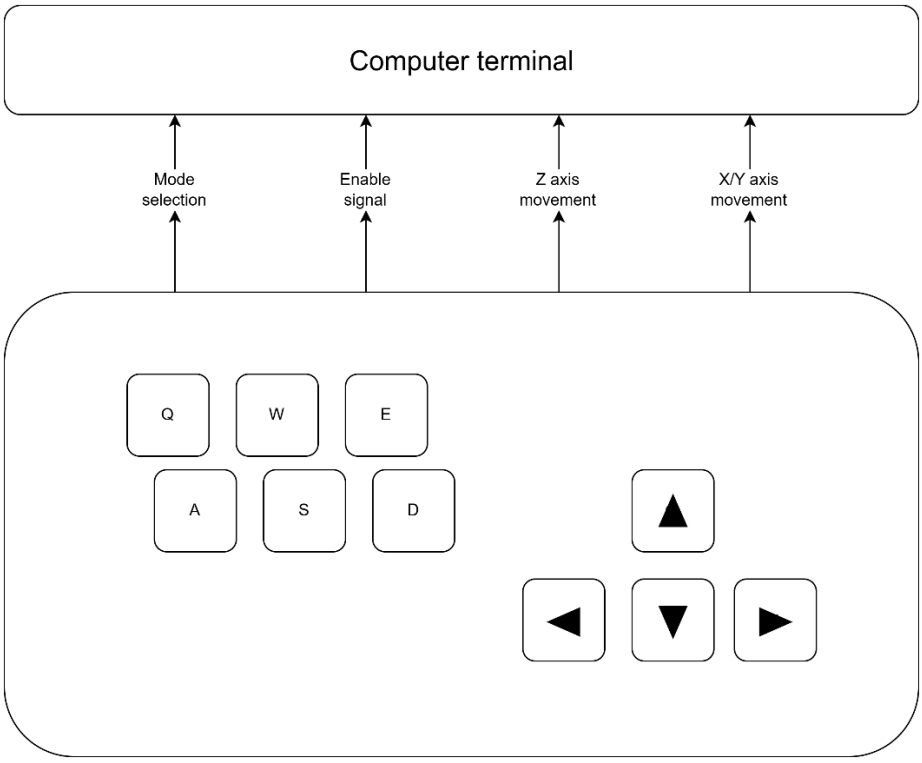


Figure 10 - Block diagram of keyboard UI.

6.7 Final Detailed Design

The final design for our system still consisted of 3 separate motor drivers and the Elegoo MEGA 2560 R3. Using software, we implemented a motion profiling PI controller and a Graphical-User-Interface (GUI) to allow the user to manually control the SCORBOT. The main components for our final design can be found below.

Final Schematic

The final schematic for our device that we developed is shown in Figure 11. This schematic consists of the Elegoo MEGA2560 R3 evaluation board, the Adafruit DRV8871 motor driver breakout boards, and the motors and microswitches on the SCORBOT. The main source of power to the Elegoo board is through the barrel jack, where a 9 V supply is used. Since each of the motors are rated for 12 V, each motor driver is supplied with 12 V through an external power supply provided by the professor. Each motor driver is used to control their respective motors. As the motors are actuated, their encoders produce square waves that can be easily read by the Elegoo board using its hardware interrupt pins. With these encoders, we can track the position, velocity, and acceleration of each of the limbs of the SCORBOT. As suggested by the professor, we placed 120 Ω resistors in series with the 5 V supply for each of the encoders to not damage their internal LEDs, and used 10 kΩ pull-up resistors to pull the encoder channels up to 5 V. As a final precaution, the Elegoo board constantly monitors the current passing through each of the motors of the

SCORBOT. Further analysis of the current measurement system is shown in the *Current Measurement* section. The Elegoo board is equipped with Microchip's ATmega2560. The Adafruit DRV8871 can drive 3.6 A with a motor voltage ranging from 6.5 V to 45 V. These specifications meet our motor requirements of 12 V at 2 A. Additionally, the digital pins of the motor driver are rated for up to 5.5 V, which suits the 5 V output from our Elegoo ATMega2560 R3 evaluation board.

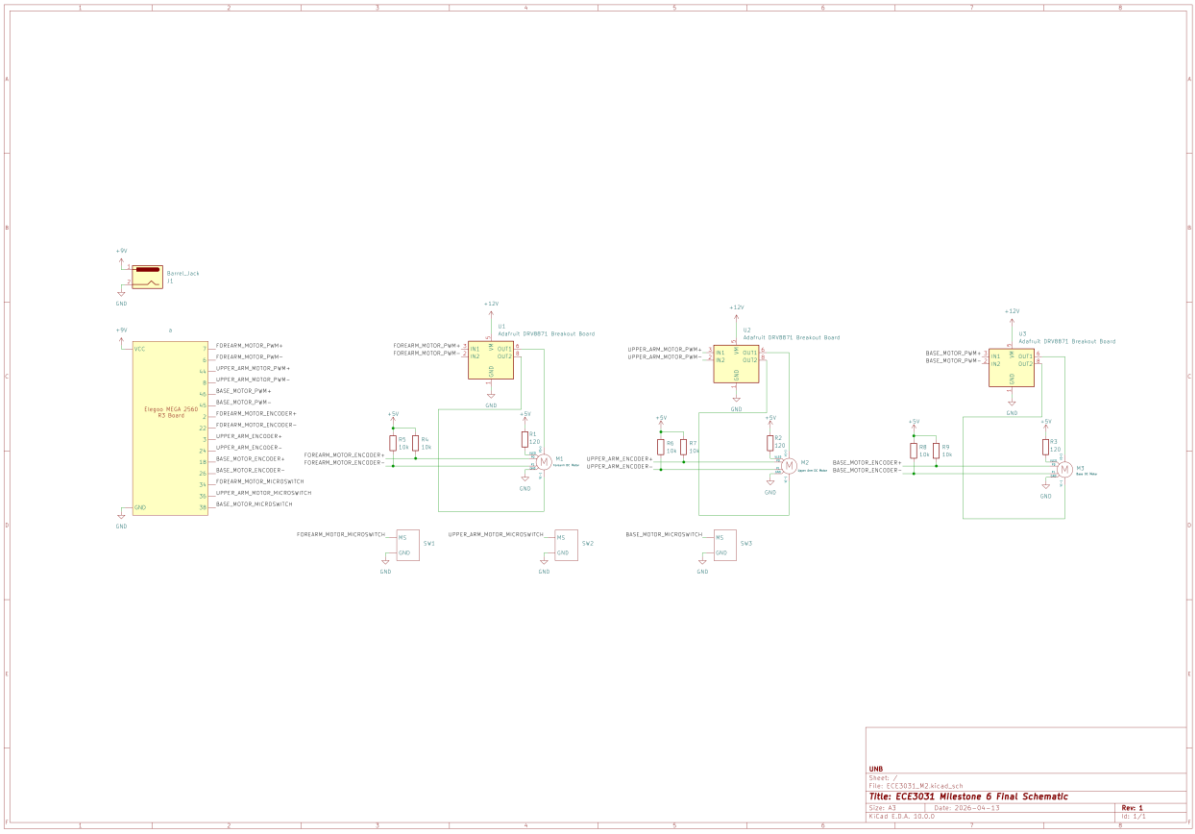


Figure 11 - Final Schematic

Controller

Every time the motor moves, the encoder on the motor produces pulses on the encoder A and encoder B pins. Using interrupts on the Elegoo board, we can determine when these pulses on the encoder pins happen. When the interrupt is triggered, an Interrupt Service Routine (ISR) is executed, and depending on the state of the encoder B pin, we can determine whether the motor is spinning in a 'forward' or a 'reverse' motion, as summarized in Table 6.

Table 6 - Summary of relation between Encoder A and Encoder B to the motor direction.

Encoder A Edge	Encoder B State	Motor Direction

↑	0	Reverse
↑	1	Forward
↓	X	X

If the motor is spinning in the reverse direction, as determined by the ISR, we subtract from our position variable. If the motor is spinning in the forward direction, we add to the position variable. This allows us to keep track of the angular position accurately of the motor relative to its starting position. To determine the angular velocity and RPM of the motors shaft, we used timer interrupts. Using timer interrupts, we set up a timer that would trigger every 100 milliseconds. The reason for the 100 milliseconds timer is to allow time for computing all of the updates for the PI controller before the timer interrupt would trigger once again. By counting the number of encoder pulses that occurred over PI update period, we can determine the angular velocity of each of the motors in the SCORBOT. Knowing that the gear ratios of each of the motors on the SCORBOT are 127.7:1 and that the number of slots in the encoder wheel is 20, we can determine that there are 2554 Clicks-Per-Revolution (CPR) of the upper arm and the forearm of the SCORBOT. The gear ratio for the base motor, however, was, from our observations, not consistent with the value listed in the datasheet. Thus, to determine the CPR of the base motor, we rotated the base motor by approximately 90 degrees, while measuring the encoder pulses, which allowed us to determine that there were approximately 4156 CPR of the base motor. With the ability to monitor the angular speed of the motor setup, we developed a PI controller to continuously monitor the motors' speed and move it towards our setpoint speed. The implementation of the PI controller was quite simple to implement, especially with the vast number of resources available online for software implementation, which helped provide us with a standard way of implementation and providing us with the basic understanding we needed so we could continue to incrementally improve on the implementation. Some examples of this are [8], [9], and [10]. However, at this point, we could only control the angular velocity of the motors on the SCORBOT. To implement controls that allowed us to control the position of the SCORBOT's limbs, we implemented a motion profiler. The general idea behind that motion profiler is that it takes a position setpoint, and based on the error between the position setpoint and the measured position that was determined using the encoders, the velocity is continuously updated such that it follows a trapezoidal profile, i.e., the velocity smoothly increases linearly, maintains a maximum, and then smoothly decreases to zero linearly, as the measured position approaches the setpoint position. There are many reasons why one would use a position-velocity-based controller with a motion profiler for the velocity instead of just a position controller. The first reason is that a purely position based controller does not limit the acceleration, and therefore the jerk applied by the motor to the arm of the robot. A position controller that does not limit the velocity and acceleration will result in a 'jerky' motion that will cause the motors that control the joints to draw a lot of current, potentially damaging the windings in the motor and causing them to burn out. With a motion profiler, the movement of the arms of

the robot is much 'smoother', preventing any sorts of damage like this from occurring. A flow diagram of the final implementation of the PI controller is shown in Figure 12. To first test our code, we used it on a single motor. After verifying its success with a single motor, we simply copy and pasted the code such that we could control all three motors simultaneously, which is further investigated in the *Multi-Motor Integration* section. Using the motion profiler, we successfully controlled the limbs of the SCORBOT in a smooth manner, as shown during our presentation. The code for the motion profiling velocity PI controllers is shown in Appendix C.

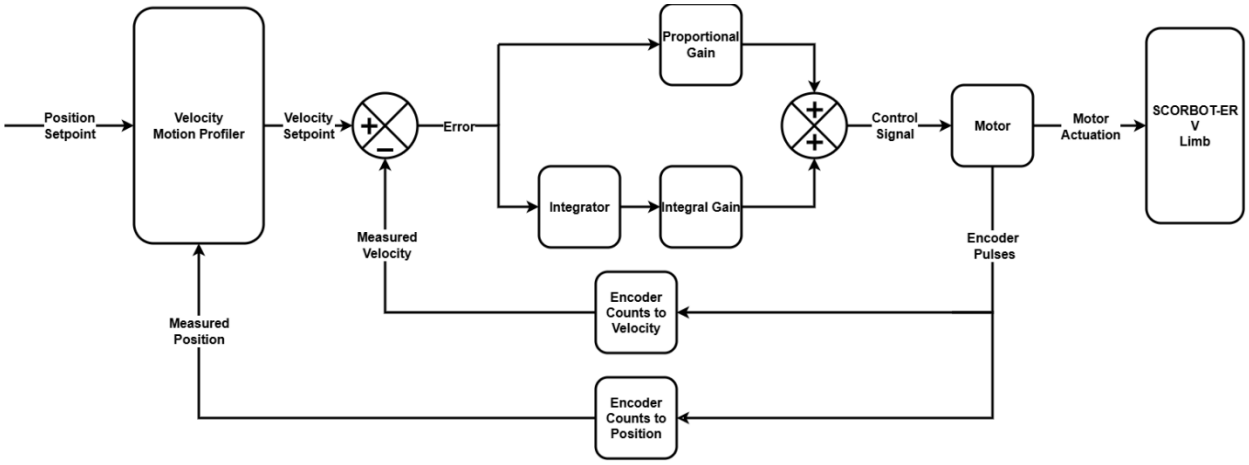


Figure 12. Final flow-diagram of motor controller.

Multi-Motor Integration

The main goal of simultaneous motor control was to convert the movement of the arm motors from degrees into an x and y coordinate system to assist in making the movement user friendly. By initially ignoring the base motor we can model the arms of the SCORBOT-ER as seen in Figure 13, where r is the radius and h is the height of the gripper, not including the end-effector. The r and h values are the desired x and y positions, which we have converted to the angular positions θ_2 and θ_3 to control motors 2 and 3, respectively. Given that r and h are the desired and therefore known values, we start with (11) and (12) to find both A and θ_L, the angle between the normal and line A.

$$A = \sqrt{h^2 + r^2} \tag{11}$$

$$\theta_L = \tan^{-1}\left(\frac{h}{r}\right) \tag{12}$$

The beams labeled B and C in Figure 13 are the arms which both have a static length of 0.22m. Knowing the length of these arms allows us to use the sin and cosine laws respectively to find the angles labeled by the lowercase parameters a and c as seen in (13) and (14).

$$a = \cos^{-1}\left(\frac{2 * 0.22^2 - A^2}{4 * 0.22}\right) \tag{13}$$

$$c = \sin^{-1}\left(\frac{L * \sin(a)}{A}\right) \tag{14}$$

Finally, we can use the angles a and c to find the angles θ_2 and θ_3 which are the angles of motors 2 and 3 respectively. These final calculations can be seen in (15) and (16) noting that the angle θ_L is a negative value hence the addition observed in (15).

$$\theta_2 = c + \theta_L \tag{15}$$

$$\theta_3 = 180° - a - \theta_2 \tag{16}$$

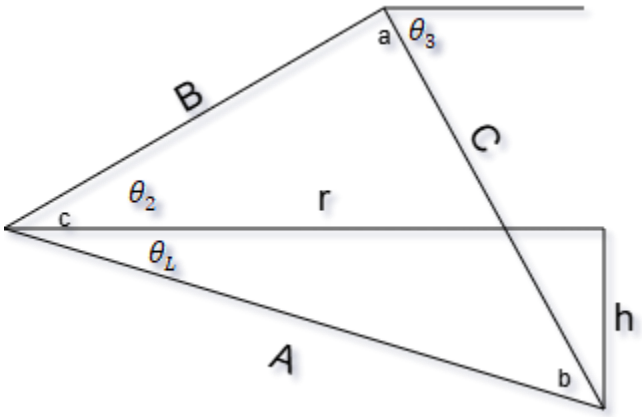


Figure 13. System model used for Cartesian coordinate conversions.

Current Measurement

Our group decided that it would be beneficial to have the option to measure the current that our motor is drawing. The simplest way to do this is to place a measurement resistor somewhere in our circuitry and use the ADC on the Elegoo board to measure the voltage across it, then calculate the current using the known resistance.

There were a few problems with this, mainly that the current would not be constant due to the PWM input of the motor, and that the voltage will be a small value. To fix this, we used a second-order low pass filter to measure the voltage across the resistor and get the DC component.

This resistor will be connected between the ground of the Adafruit DRV8871 breakout board and the ground of the Elegoo board. If the maximum current the driver will provide is set at 2A, then the maximum voltage across a 0.25 Ω test resistor would be 0.5 V. To amplify this and take advantage of the full range of the ADC, we had decided on 8x amplification. We wanted our cutoff frequency to be less than our PWM frequency, but still fast enough to not slow down the circuit.

For each motor driver, we had designed a filtering circuit that would feed into the Elegoo board's ADC. Each circuit will use two op-amps, one for filtering, and another for amplification. The filtering op-amp has unity gain, and the amplification op-amp has a gain of 7.8x, close to our target value. This gain should give us a large range of voltages for the ADC to measure while staying away from the rails of our op-amp. Our final cutoff frequency was ~450 Hz.

The frequency response of our final circuit can be seen below in Figure 14. The response to a 4 kHz square wave can also be found in Figure 15. Time response of 50% duty cycle 4 kHz square wave. This is the scenario with the largest variance in output signal, with lower or higher duty cycles resulting in a more accurate average.

It was beneficial to do this processing using an analog filter, as it means that the more of the Elegoo's computational resources could be allocated for controlling the arm. Averaging and amplifying these signals digitally would also add much more noise to the system, giving inaccurate information.

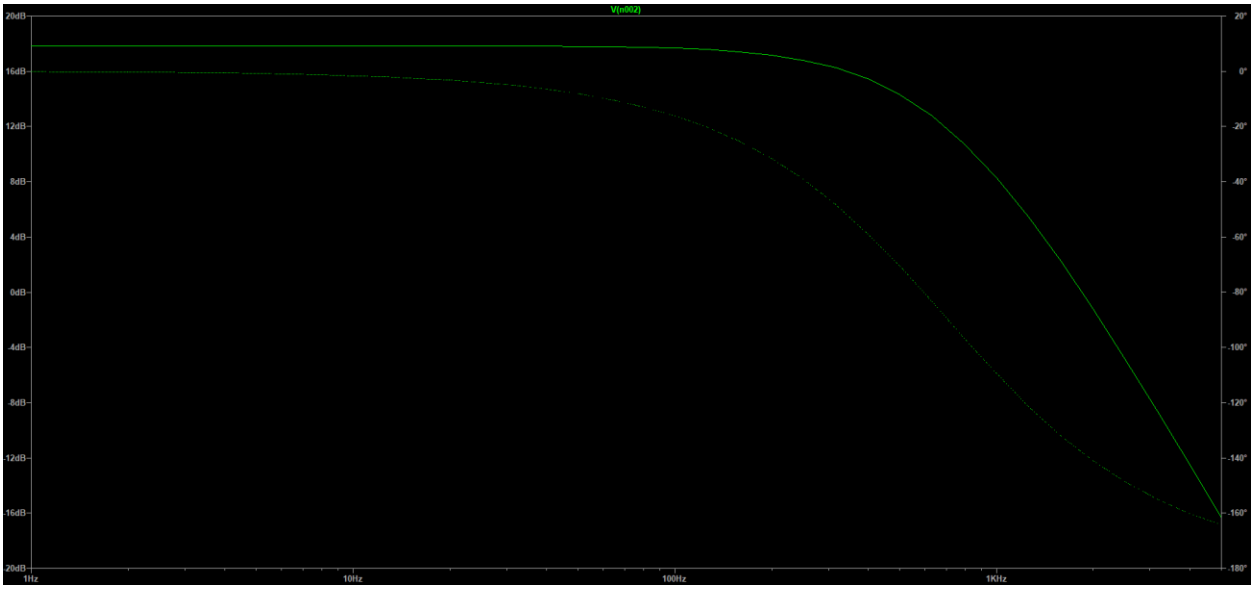


Figure 14. Frequency response of LPF circuit.

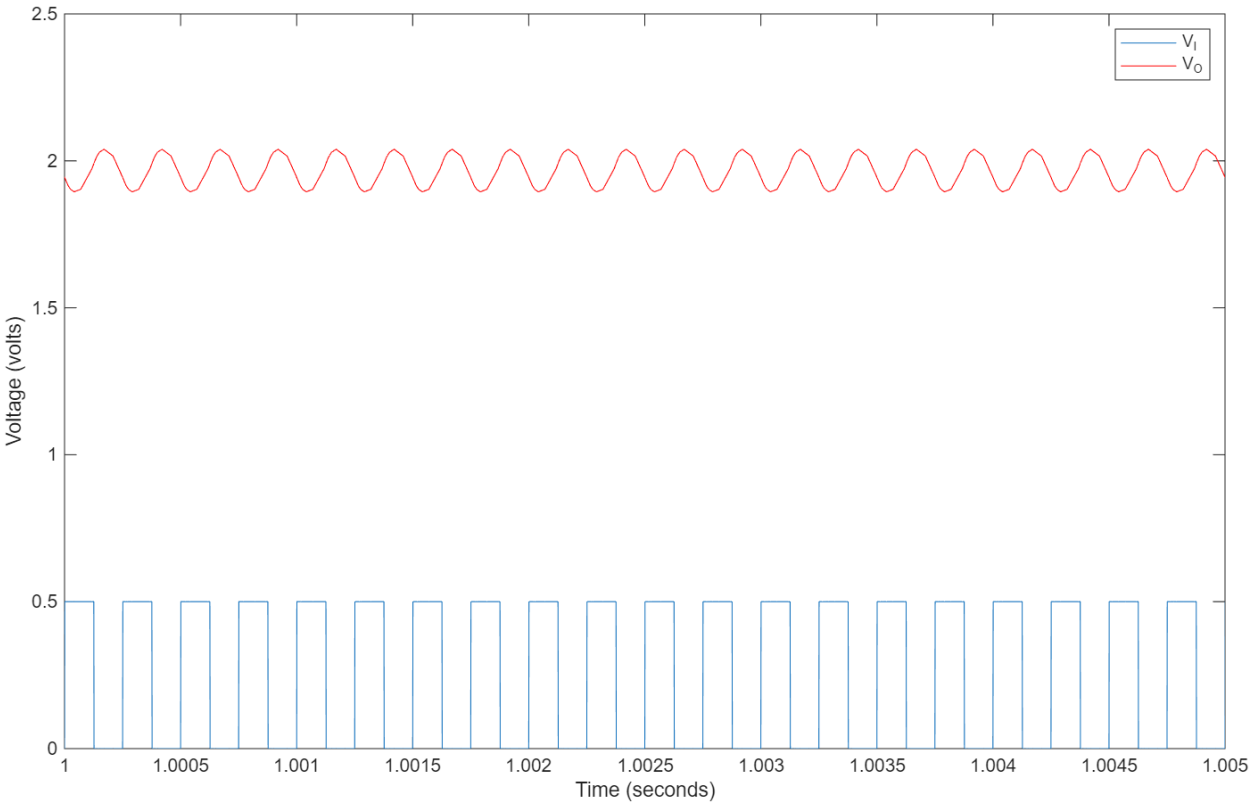


Figure 15. Time response of 50% duty cycle 4 kHz square wave.

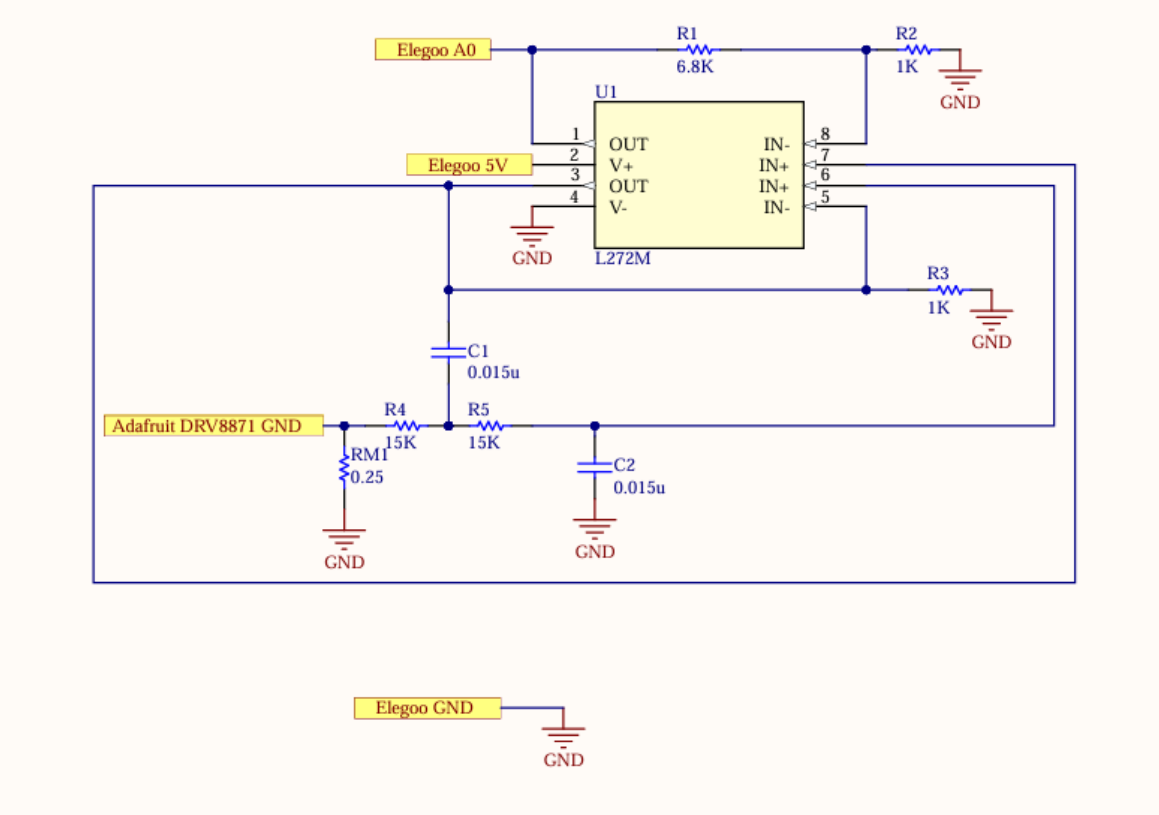


Figure 16. Schematic of current measurement circuit. This measures one motor's current, so to measure all motors, so 3 copies of this circuit will be required.

At this point in the project, we did not know if we need to measure the current, so this component of the design was done just in case. The encoders' feedback is enough to control each motor, but current measurement does give us valuable information about parameters such torque, making this a beneficial but not necessary addition to our design.

Graphical User Interface

The GUI is used to communicate with the SCORBOT-ER and send it specific commands. Instead of using LVGL, we opted to use Microsoft's Visual Studio. The completed GUI can be seen below in Figure 17.

The GUI will provide information on the status of the SCORBOT's motors by displaying position, and RPM and acceleration numbers for each motor. These buttons are also be accessible using the computer keyboard. Each motor has its position, speed, and acceleration monitored, and these are displayed in the appropriate boxes. The buttons below a set of boxes are for changing the position of that specific motor, to give the user manual control over the SCORBOT. There is also a box where a user can set the exact angle that they want the SCORBOT to go to. Each of these motors can only go in two directions, so there are two buttons. There are also arrows to control the position of the gripper. The buttons at the bottom of the window 'Home', 'Diagonal line', and

'Obstacle' perform self-explanatory tasks that facilitate the engineering and customer requirements.

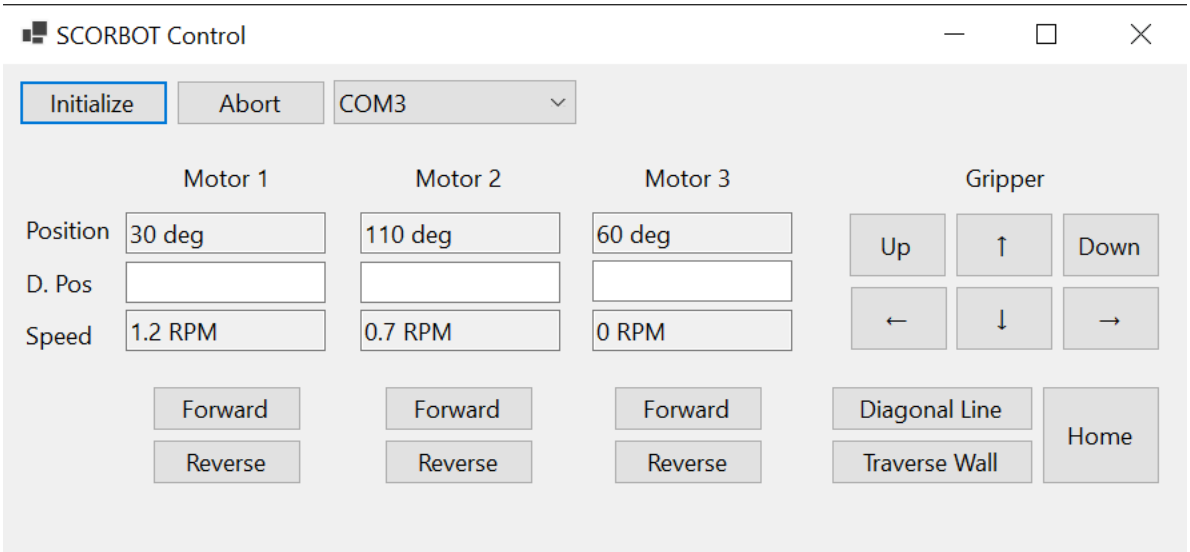


Figure 17. GUI for SCORBOT.

We used a Windows Form to make the companion app, coded in C#. To add functionality to the app, we added various event handlers for when buttons or keys are pressed. The layout stayed mostly the same as in the last iteration of the design. Control still exists for moving each motor and the gripper, as well as commands for performing things such as the diagonal line, barrier, and homing functions. Implementation was made much easier with the Visual Studio IDE. This let us drag and drop elements into our form. Without the IDE, we could have still implemented everything with the .NET framework, it just would have been more difficult.

To do this, we used the *System.IO.Ports* library to get the pre-built *SerialPort* class. One can use this to read and write to the serial port of a computer, just as the Arduino IDE's Serial Monitor does. To accommodate this, functionality has been built-in to change the serial port the program uses. It should be noted that the Arduino IDE and our app cannot be connected to the same serial port, so the Elegoo board must be programmed, and then the Arduino IDE closed before trying to connect our app to the Serial Port. The serial port can be closed and re-opened from within the app, using the 'Initialize' and 'Abort' if changes need to be made to the code.

We send/receive strings of text with different prefixes and suffixes, depending on what needs to be sent. The prefix will always be 3 characters, then there is a separating colon (:) and then a suffix. The colon and fixed prefix length lets us slice the string into its constituent parts. A summary of these can be seen in Table 7 and Table 8.

Looking at the Elegoo board, data is received with the *Serial.ReadString* command and sent with *Serial.println*. In Visual Studio, we used the *Write* method to send data and *ReadLine* method to receive data.

Table 7. Commands the app can send to the Elegoo board.

Function	Description	Prefix	Potential Suffixes
Motor A position	Move the desired motor forward or backwards by a set amount	mAs	fwd – rotate motor forward rev – rotate motor backwards
Gripper B-coordinate position	Move the gripper in the desired cartesian direction, B	gBs	inc – increase coordinate dec – decrease coordinate
Specific command	Performs a specific command	cmd	diag – move gripper in a diagonal line obst – move gripper over obstacle home – move SCORBOT arms to home position

Table 8. Information that the Elegoo board sends to the app.

Function	Description	Prefix	Suffix/Information
Motor A position	Contains the position of a motor in degrees.	mAp	The position of the motor
Motor A velocity	Contains the velocity of a motor in radians/second	mAv	The angular velocity of the motor

Motor Driving

The schematic shown for motor drivers is shown in Figure 11. This schematic consists of the Elegoo MEGA2560 R3 evaluation board, the Adafruit DRV8871 motor driver breakout boards, and the motors and microswitches on the SCORBOT. The main source of power to the Elegoo board will be through the barrel jack, where we will supply 9 V. Since each of the motors are rated for 12 V, each motor driver will be supplied with 12 V through an external power supply provided by the professor. Each motor driver will be used to control their respective motor. As the motors are actuated, their encoders will produce square waves that can be easily read by the Elegoo board using its hardware interrupt pins. With these encoders, we will be able to track the position, velocity, and acceleration of each of the limbs of the SCORBOT. As suggested by the professor, we will place 120 Ω resistors in series with the 5 V supply for each of the encoders to not damage their internal LEDs. As a final precaution, the Elegoo board will constantly monitor the status of the microswitches to determine if the SCORBOT has reached the limits of one of its axes. If the controller sees that one of the microswitches has been actuated, it will stop actuating the motors.

Overall, this design has proved to be very reliable in that, every time we connected our design to the SCORBOT interface board, we were able to repeatedly control the motors in the desired manner. The design we implemented is highly manufacturable as it requires a minimal number of components that are cheap and easy to obtain through industry standard distributors, such as DigiKey. Additionally, since the controller for the SCORBOT is implemented all in code using the Arduino platform, the design can be easily ported over to other Elegoo ATMEGA2560 boards, given that the pinouts are the same. Maintenance was a large aspect of our design. Having a design that required a lot of maintenance meant that costs due to labour, component sourcing, and re-design would go up significantly, thus, it was an important part in our design. Given that our motors, and thus the SCORBOT's limbs, operate smoothly when transitioning from point-to-point, the wear-and-tear on the motors was reduced significantly. This meant that our design could last very long before needing maintenance of any sort. Given this, our design is highly maintainable and cost in terms of labour, component selection, and re-design is very low. A summary of component costs for our project are shown in Table 9. Note that while some of these components were provided to us, we attempted to find similar components on DigiKey such that the true cost of our design was accurately reflected if someone wanted to reproduce our design. A summary of all equipment and part numbers for each component used is shown in Table 9.

6.8 Final Component Selection and Bill of Materials

The final bill of materials for our final design is shown in Table 9. Note that this table does not include the SCORBOT itself and the motors used for testing since these were not in the final implementation of our controller.

Table 9. Bill of materials for our final design.

Description	DigiKey Part Number	Manufacturer	Manufacturer Product Number	Quantity	Unit Price ($)	Total Price ($)	Reference Designators	Footprints
Motor Driver	1528-3190-ND	Adafruit Industries LLC	3190	3	10.83	32.49	U1, U2, U3	Breakout board
Encoder Resistors	13-MFR-25FTE52-120RCT-ND	YAGEO	MFR-25FTE52-120R	3	0.10	0.30	R1, R2, R3	Through-hole
Power Resistors	WHCR25FECT-ND	Ohmite	WNCR25FET	3	2.69	8.07	RM1	Through-hole
Op-amps	L272MFS-ND	onsemi	L272M	3	1.14	2.28	U1	Through-hole
Capacitor, 15 nF	445-180802-1-ND	TDK	FA28X7R1H153KNU06	6	0.39	2.34	C1	Through-hole
Resistors, 15 kΩ	CFM14JA15K0-ND	Stackpole Electronics Inc	CFM14JA15K0	6	0.01351	0.08106	R4, R5	Through-hole

Resistors, 1 kΩ	CFM14JA1K00-ND	Stackpole Electronics Inc	CFM14JA1K00	6	0.01351	0.08106	R2, R3	Through-hole
Resistors, 6.8 kΩ	CFM14JA6K80-ND	Stackpole Electronics Inc	CFM14JA6K80	3	0.01351	0.04053	R1	Through-hole
Resistors, 10 kΩ	13-MFR-25FRF52-10KTR-ND	YAGEO	MFR-25FRF52-10K	6	0.15	0.90	R4, R5, R6, R7, R8, R9	Through-hole

6.9 Health, Safety, Environmental, and Social Considerations

The two primary concerns in a mechanical project, such as the control system of the SCORBOT-ER, are the safety of the user and the safety of the system itself. The main concern is the safety of the user which we addressed by limiting the maximum speed of the motors, making use of the microswitches, PWM-based limitations, and careful testing procedures. By having the motors move at a slower speed, harmful situations are far easier to prevent, as it provides the user with time to react. The limitation of speed is also a strong method to help prevent damage to the system as the slower speed minimizes the jerk of the motors when they begin moving on top of providing the user with the necessary time to shut down the system when things don't go to plan.

Electrical safety was also considered, as the motors used in the SCORBOT-ER are rated up to 12 volts. The user-selected current limitations of the supply and the protection circuitry integrated on the motors drivers reduces the risks of component damage due to electrical faults.

Minimizing material waste was an important environmental consideration. Through the our initial research, several key parameters were identified to guide us in the selection of specific components. Had we not carefully evaluated certain parameters such as the required motor current, it is likely that we would have selected incompatible components resulting in wasted time, materials, and additional shipping. Furthermore, smoother motor movement reduces the amount of wear on the motors, resulting in less frequent replacing of the motors. By ensuring smooth acceleration and deceleration, we can reduce the frequency of part replacement and in turn minimize the long-term waste.

Constructing a user-friendly and smooth control system has numerous social applications outside of controlling a robot to perform specific tasks. In our specific design, having the robot move at a controlled speed allows the user or other individuals the necessary time to avoid getting pinched between the arms of the SCORBOT-ER. The software shut off, along with the numerous mechanical shutdowns, allow the user to halt movement for any case where a person is at risk of injury or the robot itself is at risk of damage. These design practices promote safe operation not only within our design but also through other real-world systems where reliability and safety are crucial.

6.10 User Guide

The main way that the SCORBOT can be controlled is via the GUI application that we have made, which can be seen in Figure 17. To get started, program the Elegoo board with the firmware if it hasn't been already, then open the GUI application, *3031Project.exe*. Make sure the Arduino IDE is closed when doing these next steps. Go into your computer's device manager and see which COM port the Elegoo is connected to under the 'Ports' heading. Once this is completed, select the appropriate COM port on the Elegoo Board and click the 'Initialize' button.

The SCORBOT controller is interfaced to the SCORBOT via an interface connection board, which allows the Elegoo board to communicate with the SCORBOT by receiving encoder pulses and sending out PWM signals through the external driver boards. Additionally, these driver boards require additional supply power, 12 V and ground as its power rails, so that it can sufficiently power the motors on the SCORBOT. To successfully connect the SCORBOT to the Elegoo board, there is a SCORBOT interface connector to connect between the SCORBOT and the interface board. This connection must be ensured to successfully communicate between the controller and the SCORBOT. A schematic showing the necessary connections to the SCORBOT are shown in Figure 11.

Once the SCORBOT is connected, the position and angular velocity of each motor can be seen. To move a motor, one can click the respective 'Forward' or 'Reverse' buttons to increase or decrease the position of the particular motor by approximately 20 degrees. The 'D. Pos' boxes can also be filled in, which sets the angle of the motors when the box itself is double-clicked.

The position of the gripper can also be controlled using the up and down arrows within the GUI. Along with this, the functionality of the other buttons is listed below.

- Home – Returns SCORBOT to its home position.
- Traverse Wall – Moves the gripper over the wall from the project definition.
- Diagonal Line – Moves the gripper in a diagonal line.

In addition to these functions, users can also view the speed of each motor in the system. This is important to note, because the SCORBOT's environment should be taken into consideration, and speed must be maintained at a lower level when in close proximity to obstacles or otherwise.

Users must be aware that the SCORBOT must be reset after each set of operations such that it is manually returned to its 'home' position. This position consists of the SCORBOT being pushed to its limits, all the way backward for the upper arm motor, all the way downward for the forearm motor, and spun to the limit switch for the base motor. This will look like the SCORBOT is 'folded' upon itself, and the arms are in contact with each other. This is the baseline location for the SCORBOT to operate from, and resetting to this position ensures optimal performance and minimal error within the system, which is critical to the success of the controller. In addition to the physical reset, users should also fully reset the software to ensure that all encoder counts return to

zero, ensuring that the controller recognizes that it is back in its home position. For maintenance of the SCORBOT and controller, ensure that wires connecting the controller to the SCORBOT interface board do not become unplugged, as this may result in erratic behaviour. Additionally, ensure that no wires on the board are shorted together as this may damage the controller and/or the SCORBOT.

References

[1] Crane Tech Solution, “Level Luffing Crane,” Crane Tech Solution. Accessed: Feb. 14, 2026. [Online]. Available: https://cranetechsolution.com/level-luffing-crane/

[2] L. Edwards, *Open-Source Robotics and Process Control Cookbook: Designing and Building Robust, Dependable Real-Time Systems*. Newnes, 2005.

[3] Eshed Robotech, “SCORBOT-ER Vplus User’s Manual.” Eshed Robotech, Israel, Oct. 1998.

[4] onsemi, “NCP51530.” onsemi, Feb. 2023.

[5] Performance Motion Devices, “Magellan® Family of Motion Control ICs.”

[6] Texas Instruments, “LM628/LM629 Precision Motion Controller.” Texas Instruments, Jun. 1999.

[7] L. Fried, “Adafruit DRV8871 Brushed DC Motor Driver Breakout Datasheet.” Adafruit Industries, 2024.

[8] DC motor and PID speed control, https://www.youtube.com/watch?v=HRaZLCBFVDE, Curio Res, 2021.

[9] Motion Profiling, https://rr.brott.dev/docs/v0-5/tour/motion-profiling/, Brott

[10] Smoothstep, https://en.wikipedia.org/wiki/Smoothstep, Wikipedia

Appendix A: Test & Verification Plan

A verification plan is a critical component of our design process to ensure that our design functions and meets the requirements developed between the customer and ourselves. The following is a list of verification plan steps, labelled '**VP-X**', that we have developed to ensure functionality of our design according to the customer and engineering requirements:

VP-1: This test is a verification of the structural integrity of our design. We must ensure that the SCORBOT-ER remains intact and provides enough torque to move the motors while still being structurally intact. This task is successful if the SCORBOT-ER can successfully move safely and without damage. More sensitive components in the SCORBOT-ER's construction are the priority of this test and should be safeguarded. If this test is not successful, we must immediately reiterate through the design of our system before proceeding with the following verification plans.

VP-2: This test is for verifying the functionality of the SCORBOT-ER autonomous line drawing system. For this test, will first draw two, 500 mm lines on a wall at an angle of 45° and a spacing 20 mm between them. We will then generate the desired path in code and feed it into our PID controller and place a marker into the gripper of the SCORBOT-ER. If the control system is able to keep the line drawn by the marker attached to the gripper of the SCORBOT-ER within the 10 mm tolerance, we consider this test successful in measuring the effectiveness of the autonomous line drawing system. If the line drawn is not within the 10 mm tolerance, we will need to reiterate through the design of our control system.

VP-3: This test is for verifying the functionality of the SCORBOT-ER autonomous wall traversal system. For this test, the wall will have a height of 27 cm, while the platform behind the wall will have a height of 13 cm. The horizontal traversal of the arm should be 25 cm. This test will include the wall and platform and will be considered successful if the arm touches the platform after traversing 25 cm horizontally and avoiding the wall. After touching down on the other side of the wall, the arm must return to the original location avoiding the wall a second time. For this test, we are looking to see that the arm is stable, moves at a reasonable pace, and can accurately move where it must move to with minimal error. If the SCORBOT-ER is capable of traversing over the barrier, we consider this test successful in determining that the SCORBOT-ER control system can accurately control the arms to traverse over the barrier. If the SCORBOT-ER cannot traverse over the wall, we will need to reiterate through the design of our control system.

VP-4: Manual control is to be tested in this verification step. To verify the function of this requirement we will ensure that the manual mode can be selected, and then the SCORBOT-ER can be manually operated via a keyboard, or other input means. To verify that this is successful, we must ensure that the device can respond to user inputs with promptness and minimal error, as well as each input mapping to the correctly intended output. Additionally, we must ensure that there is not excessive torque accumulation or dangerous speeds when operating in manual control mode.

VP-5: Sufficient power supply is the goal for this test. To confirm that this is successful, we must ensure that the controller and all motors have sufficient power supplied to allow for the required torque and successful operation. Loss of power to any degree is considered a failure of this test, whether it is momentary or sustained power loss.

VP-6: To confirm the functionality of the PWM motor control the SCORBOT-ER will be placed into manual operation, and the position will be varied. The effectiveness of our PWM control will then be monitored through the speed of the motors, ensuring that we can achieve the desired speed.

VP-7: This verification is in place to confirm that with all other criteria met, the control of the SCORBOT-ER is performed using the Elegoo MEGA board. Other UI shall be used but the main control on the motors, including the PI controller, shall be loaded onto and performed through the provided Elegoo MEGA board.

VP-8: The system should normally operate well below the maximum safe speed for operation however, we will add a fail-safe for the extreme case. To test it, the motors will be operated as fast as possible in each mode of operation where the angular velocity will be found through the feedback system. The controller will be adjusted until the speed requirement of ~~30~~ 120 degrees per second is met.

VP-9: As of now the cost requirement is not a concern as we are well within the $50 dollar limit, however we should continue to monitor the budget where additional expenses are incurred.

A summary of the verification plan as it applies to the engineering requirements can be found below in Table 10.

Table 10 - Summary of tests to meet the engineering requirements.

Relevant ER #	Test	Passing criteria	Pass / Fail / N/A
ER-1	Connect Control system to the SCORBOT-ER and test movement as well as its ability to resist disturbances such as varying loads.	Smooth consistent movement of each motor with no damage to the SCORBOT-ER.	This test has passed.
ER-2	Place the SCORBOT-ER in its autonomous line mode set to move 500 mm at 45° and monitor the deviation from its path.	Successful tracing of the line with a deviation of less than 10 mm from its intended path.	This test is N/A since, due to lack of availability with the SCORBOT, we have not been able to perform this test.
ER-3	Try both the autonomous traversal mode and manual	Successfully elevate over a 27 cm wall, extend to 25 cm and	This test is N/A since, due to lack of availability with the

	operation to have the robot elevate over the barrier, extent to the platform and descend onto said platform.	descend to a platform at a height of 13 cm without contacting the barrier.	SCORBOT, we have not been able to perform this test.
ER-4	Move each motor individually to a specified position to confirm position control while monitoring each motor's angular velocity and acceleration.	Have the motor end within 5% of the desired position, while having the motors move smooth enough to not cause any kickback.	This test is N/A since, due to lack of availability with the SCORBOT, we have not been able to perform this test.
ER-5	Have all motors move at their highest desired velocity with the maximum expected load and make sure that the motors still move as expected	Motors still move with the expected speed and smoothness. No damage is caused to either the supply or motors.	This test has passed.
ER-6	Test all motors with a wide range of PWM and see if the speed is as expected.	Motors move at the expected speed relative to the applied PWM input.	This test has passed.
ER-7	Inspect the final setup to confirm the Elegoo Mega 2560 is being used in the control system.	If the final control system product implements the use of the Elegoo Mega Board then the test is successful.	This test has passed.
ER-8	Ensure that no motor exceeds a rotational velocity of 120 degrees per second.	If any motor rotates at an angular velocity of 120 degrees per second or greater, the test is considered failed.	This test has passed.
ER-9	Ensure that all components utilized in the design of the controller are under the specified cost of $50.	All components in the fully built control system cost less than $50.	This test has passed.

For the testing and verification plan, we defined each subsystem as an individual motor. Both of the motors that control the upper arm and the forearm of the robot will receive the same testing. The testing plan for the motors that control the upper arm and forearm of the robot are shown in Table 11 and Table 12. The testing process for the final subsystem, the base motor, can be found in

Table 13 with some of the tests being very similar to the other motors, and some quite different due to the differences in their purposes.

Table 11 – Upper arm verification plan.

Item	**Procedure**	**Expectation**	**Result**	**Pass / Fail / N/A**
Ability to Actuate	We must connect the PWM output of the Elegoo board into the driver board, as well as driver power, which is a 15V and ground line. Once all connections are made, we must ensure that the signal that is sent from the Elegoo board to this motor's driver board. The PWM can be sent using the keyboard through the serial monitor on the Arduino IDE.	The motor must be able to move in both directions, clockwise and counterclockwise in response to the PWM input, and must be proportional to the input.	Successful actuation of the motor.	This test has passed.
Stress Test	We will test the minimum PWM input signal required to cause each individual motor as well as the maximum desired to limit the speed. This will be done by slowly increasing the PWM from 0 util we see movement. For the maximum we can again increase the upper limit of the PWM signal until the motor is moving at a speed comfortably within our pre-determined safe range.	We expect that the motors will require an increase in PWM before the motors generate enough torque to move the arms. This value should be recorded. Then we will also measure the maximum PWM signal to the motors by setting a constant velocity of 120 degrees per second and determining the power draw at this point of the operation.	N/A	This test is N/A since, due to lack of availability with the SCORBOT, we have not been able to perform this test.

Integration with Multiple Motors	We will test the system's ability to integrate multiple motor movements together into one output movement, such as moving the gripper in a straight line, or through manual movement. These commands will be given through the GUI.	The expected result for this test is that the motors will operate simultaneously to achieve the expected position and/or velocity that the system should achieve. This motor is responsible for the Θ_2 angle with respect to the horizontal plane.	System integration with multiple motors was successful.	This test has passed.
Error Accumulation	This test is meant to measure the accumulation of errors within the motors as they move from position to position. To complete this test, we will give the control system many movement commands of any variety and determine if the error within the system increases over time.	We expect that the error accumulation will be minimal early on, but as the system continues to run, we expect to see some deviation from nominal. We will be looking to ensure that the final error after running the system for an extended period will be minimal.	The final result accumulated approximately 40 degrees of error.	This test has failed due to the large error that accumulated as the SCORBOT operated.

Table 12 - Forearm verification plan.

Item	**Procedure**	**Expectation**	**Result**	**Pass / Fail / N/A**
Ability to Actuate	We must connect the PWM output of the Elegoo board into the driver board, as well as driver power, which is a 12V and ground line. Once all of these connections are made we must ensure that the signal that is sent from the Elegoo board	The motor must be able to move in both directions, clockwise and counterclockwise as a response to the PWM input, and must be proportional to the input.	Successful Actuation	This test has passed.

	to this motor's driver board. The PWM can be sent using the keyboard through the serial monitor on the Arduino IDE.			
Stress Test	We will test the minimum PWM input signal required to cause each individual motor as well as the maximum desired to limit the speed. This will be done by slowly increasing the PWM from 0 util we see movement. For the maximum we can again increase the upper limit of the PWM signal until such that the motor is moving a speed that is comfortably within our safety range.	We expect that the motors will require an increase in PWM before the motors generate enough torque to move the arms. This value should be recorded. Then we will also measure the maximum PWM signal to the motors by setting a constant velocity of 120 degrees per second and determining the power draw at this point of the operation.	N/A	This test is N/A since, due to lack of availability with the SCORBOT, we have not been able to perform this test.
Integration with Multiple Motors	We will test the system's ability to integrate multiple motor movements together into one output movement, such as moving the gripper in a straight line, or manual. These commands can be given through the GUI.	The expected result for this test is that the motors will operate simultaneously to achieve the expected position and/or velocity that the system should achieve. This motor is responsible for the angle Θ_3 as seen in Figure 13.	Successful Integration	This test has passed.
Error Accumulation	This test is meant to measure the accumulation of errors within the motors as they move from position to position. To complete this test, we will give the control system many	We expect that the error accumulation will be minimal early on, but as the system continues to run, we expect to see some deviation from nominal. We will be looking to ensure that the final error after running	8 degrees	This test has passed.

	movement commands of any variety and determine if the error within the system increases over time.	the system for an extended period will be minimal.		

Table 13 - Base motor test plan.

Item	Procedure	Expectation	Result	Pass / Fail / N/A
Ability to actuate	This test is meant to ensure that the base motor can respond to PWM signals sent from the Elegoo board through the driver board and into the motor. We will supply the driver with 12V supply voltage, and also the PWM signal which the motor will rotate proportionally to.	The expectation is that this motor will actuate in response to PWM signal and respond proportionally to the duty cycle of the PWM signal.	Successful Actuation	This test has passed.
Stress Test	We will test the minimum PWM input signal required to cause each individual motor as well as the maximum desired to limit the speed. This will be done by slowly increasing the PWM from 0 util we see movement. For the maximum we can again increase the upper limit of the PWM signal until such that the motor is moving a speed that is comfortably within our safety range.	We expect that the minimum PWM duty cycle required to generate enough torque in the motor is significant, and we look to measure this value to see what the minimum PWM duty cycle is needed to move the motor at its slowest speed. Additionally, we need to determine the maximum duty cycle that we can provide while safely swiveling the base without exceeding any safety requirements.	N/A	This test is N/A since, due to lack of availability with the SCORBOT, we have not been able to perform this test.
Coordination with Arm Motors	This test is meant to ensure that the base motor is properly coordinated with the arm motors for automated tasks, namely the wall traversal task. This is important for the wall traversal since it is important that the	We expect that the base motor will act in coordination with the arm motors properly and successfully traverse the wall.	Successful Integration	This test has passed.

	azimuth angle is controlled in conjunction with the arm's attitude, so this will be tested by enabling the wall traversal mode and ensuring that the motors act together.			
Error accumulation	This test is designed to check the base motor's error accumulation. To test this we will provide many different commands to change the azimuth angle of the SCORBOT-ER, and upon completion of all commands we will determine how far from nominal the real angle is.	We expect the system total error to be nonzero, but ideally is significantly mitigated to the point where the error approaches zero.	This test plan has passed since we have not accumulated more than 1 degree of error.	This test has passed.

We were able to test the control system on the SCORBOT, and the above tables show the results of the testing. We see that there is some error accumulation as we continually reset the system, which hints to missed encoder ticks after multiple uses. To avoid this error, we realize that the system must be manually reset and verified before each usage to ensure that there is no error accumulation that could cause potential damage or overcurrent to the system if it were to collide with itself.

The final product is only expected to operate indoors, in a classroom in Head Hall. This means that the atmospheric conditions will be relatively constant. The power we have available will also be constant, as long as the Head Hall electrical system can provide constant power to the DC power supply used to power the SCORBOT. The only variable here is the USB port of the computer that the Elegoo is plugged into. In our testing, this did not have much of an effect. If it did, we could simply use the barrel plug that comes with the board.

The first thing we tested for was that we had accurate control of the position of the motors. This angular position was set in the code, and the response of the motors was observed by the team. Everything behaved as expected, with the only problem being not plugging in wires correctly. We also had to do a bit of experimentation with mapping encoder ticks to change in position, but the provided documentation helped with that.

Another thing we tested was the implementation of derivative control. In our experience, derivative control often caused violent stopping of the motors when they got close to the desired positions. Beyond this, we also attempted to integrate derivative control as a form of velocity

control, using changes in position over time in place of error differences. This caused very strange behaviors including numerous oscillations depending on the value of k_d used. Due to the challenges and unexpected behaviors when integrating derivative control, we decided to omit it, opting to use a simple PI controller which provided very smooth control on its own.

The next thing that was tested was the ability to operate multiple motors simultaneously. Similarly, to the single motor, the position of each motor was set in the code. We tested to see that each motor would rotate to the specified angle with all of the motors moving at the same time. There were no major challenges or redesigns with this, besides simple software bugs.

Finally, we wanted to be able to give the controller an absolute angle, and the motors move to the specified angle, instead of just increasing/decreasing the angle relative to where it was before. This will make it easier to control the position of the gripper, as we can directly use the angles from the math used to calculate the gripper's position, and vice versa. This had the most problems, where the motor would spin twice before arriving at the first desired position, then continue on through the requested positions accurately. The controller thought that it needed to get to the initial position provided by the code, but the motor was already at that position, which is where that first extra spin came from. We were able to fix this by adding a delay in the code before main execution. This was most likely due to the discrepancy between its actual starting position and its setpoint.

We also had to iterate several times throughout the design process for the current measurement device. The first design used a differential op-amp in combination with a small resistor to find a voltage drop, then convert that voltage to a current on the Elegoo. This was connected between the motor's voltage inputs and the Adafruit DRV8871 motor driver. This had the problems like changing voltage polarity, and a lack of filtering giving the Elegoo board a PWM signal. To improve this, we constructed a second-order filter, shown in Figure 18. However, this had the downside of being unstable when a larger amplification ratio was required, so this design also had to be modified. In the end, we settled on a second order filter connected in series with an inverting op-amp. This filter, seen in Figure 19, gave us everything we wanted, successfully averaging and amplifying the PWM signal to a point where a large range of the Elegoo's ADC could be used.

Had more time been available for us to work on the project and with the SCORBOT, we would have implemented fully autonomous controls for the SCORBOT. However, due to a lack of availability with the SCORBOT, we were not capable of testing the SCORBOT to verify if our controls would work and to demonstrate our device. Additionally, we think it would be interesting to try using a more powerful microcontroller to investigate if we could keep up with the encoder pulses from the motors.

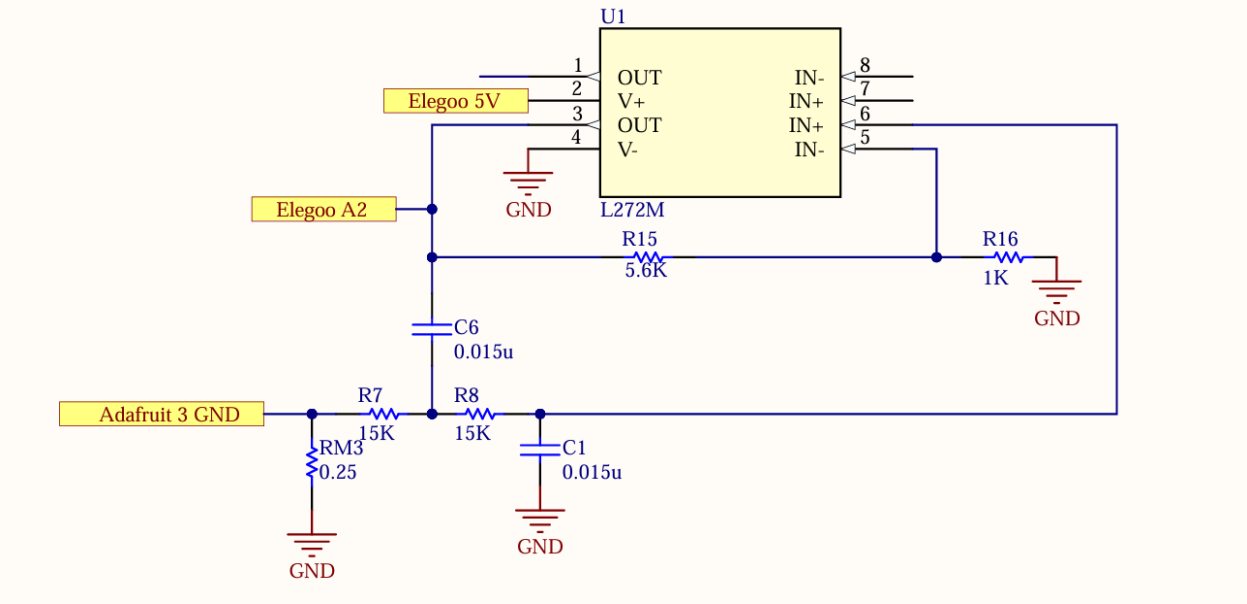


Figure 18. Second iteration of current measurement design.

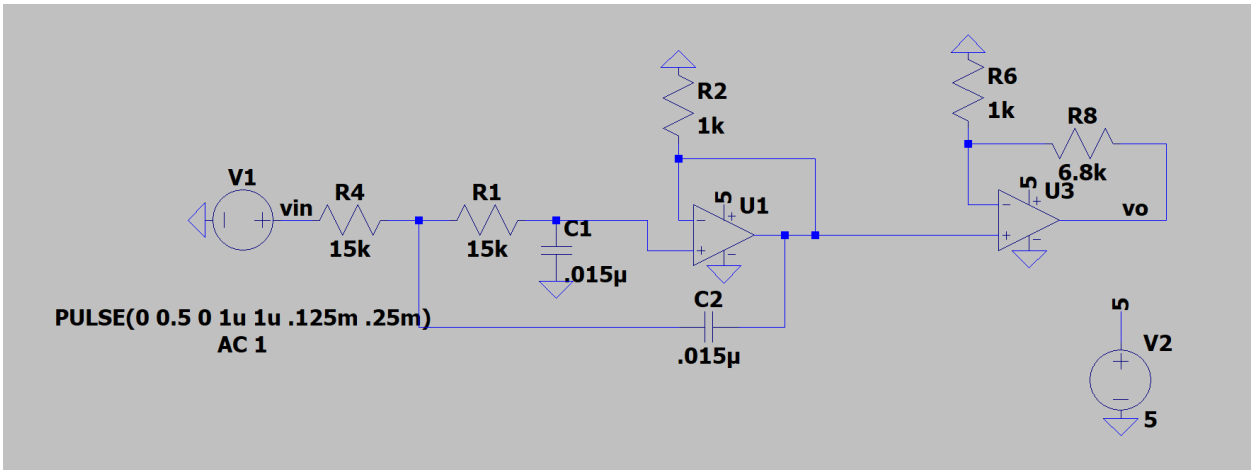


Figure 19. LTspice schematic of final current measurement design.

Appendix B: Gantt Chart

Shown below in Figure 20 is the Gantt chart for our semester project.

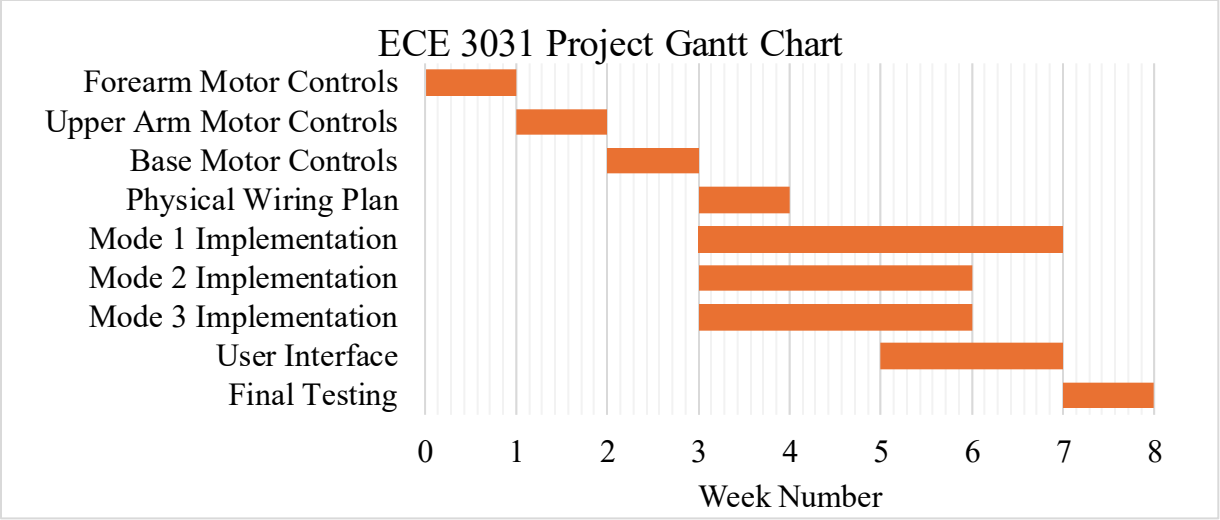


Figure 20. Project Gantt chart.

Forearm Motor Controls: This task requires the winch motor and the Elegoo board. For this task, we must develop code to operate and control the winch motor. (Estimated time: 3-4 hours per person)

Upper Arm Motor Controls: This task requires the linear actuator and the Elegoo board. For this task, we must develop code to operate and control the upper arm. (Estimated time: 3-4 hours per person)

Base Motor Controls: This task requires the base motor and the Elegoo board. For this task, we must develop code to operate and control the base motor. (Estimated time: 3-4 hours per person)

Physical Wiring Plan: This task requires the layout of the SCORBOT-ER, dimensions, and wires, as well as Elegoo board. For this task, we will design the layout of the physical setup and the wiring of the system. (Estimated time: 3-4 hours per person)

Mode 1 Implementation: This task, and all other mode implementations require all previous tasks to be complete. For this task, we must develop code to allow manual control of the SCORBOT-ER with the control system. (Estimated time: 4-5 hours per person)

Mode 2 Implementation: For this task, we must develop code to autonomously draw a straight line with the SCORBOT. (Estimated time: 7-8 hours per person)

Mode 3 Implementation: For this task, we must develop code for the control system to autonomously move the load over a barrier. (Estimated time: 7-8 hours per person)

User Interface: This task must be developed in parallel with Mode 1 Implementation. This task requires physical components, such as a 3-way switch, 2 joysticks, 2 buttons, and a liquid-crystal-display. (Estimated time: 4-5 hours per person)

Final Testing: This task implements the final testing of our design to ensure system functionality. This task requires that all other tasks are complete. (Estimated time: 4-5 hours per person)

Our original plan can be seen above. This plan was followed to some extent, but the motor controls, wiring plan, and user interface were all developed in parallel. This was because it was easiest to divide work up in this way, and all of the test motors the professor gave to us were identical. After all of the motors worked by themselves, we worked on running them simultaneously. After this was complete, we moved over to testing our controller on the SCORBOT and finalizing the user interface. At this point, we worked on implementing the barrier and obstacle tasks.

Besides the flow of the project, the time estimates were slightly off, as the earlier planning took less time than we thought, but the later design took more time than we expected.

Appendix C: Software/Firmware

Code for controller:

Code for the PI position controller (outdated, use attached files on Crowdmark as reference):

```
#define ENCODER_A 2
#define ENCODER_B 3
#define PWM_IN1 12
#define PWM_IN2 13

int32_t position = 0;
int32_t previous_time = 0;
float previous_error = 0;
float integral = 0;
float error = 0;

void setup() {
  Serial.begin(9600);
  pinMode(ENCODER_A, INPUT);
  pinMode(ENCODER_B, INPUT);
  attachInterrupt(digitalPinToInterrupt(ENCODER_A), readEncoder,
RISING);
}

void loop() {
  int32_t setpoint = 1000*sin(previous_time/1e6);

  float kp = 1;
  float ki = 1;
  float kd = 0;

  int32_t current_time = micros();

  float delta_time = (float)((current_time -
previous_time)/1.0e6);
  previous_time = current_time;

  int error = setpoint - position;

  // Derivative
```

```
  float derivative = (error - previous_error)/delta_time;

  // Integral
  integral += error*delta_time;
  // This would probably be useful for preventing integrator
windup
  // Noticed that motor is 'choppy' at the extremes with input
sinewave
  if (((previous_error < 0) && (error > 0)) || ((previous_error >
0) && (error < 0)) || (position == setpoint)) {
    integral = 0;
  }

  // Control signal
  float u = kp*error + ki*integral + kd*derivative;

  // Add max PID value and normalize it to duty cycle?

  float pwm_dutycycle = fabs(u);
  if (pwm_dutycycle > 100) {
    pwm_dutycycle = 100;
  }

  int8_t direction = 1;
  if (u < 0) {
    direction = -1;
  }

  // Makes motor choppy at extremes of input sine
  // if (position == setpoint) {
  //   direction = 0;
  // }

  setMotor(direction, pwm_dutycycle, PWM_IN1, PWM_IN2);

  previous_error = error;

  // Serial.print("Setpoint = ");
  // Serial.println(setpoint);
  // Serial.print("Position = ");
```

```
  Serial.println(position);

  // Serial.println(u);
}

void setMotor(int8_t direction, uint8_t pwm_dutycycle, uint8_t
pin_in1, uint8_t pin_in2) {
  if (direction == 1) {
    digitalWrite(pin_in1, HIGH);
    analogWrite(pin_in2, 255 - pwm_dutycycle*255/100);
  } else if (direction == -1) {
    digitalWrite(pin_in2, HIGH);
    analogWrite(pin_in1, 255 - pwm_dutycycle*255/100);
  } else {
    digitalWrite(pin_in1, HIGH);
    digitalWrite(pin_in2, HIGH);
  }

}

void readEncoder() {
  uint8_t valueRead = digitalRead(ENCODER_B);

  if (valueRead) {
    position++;
  } else {
    position--;
  }
}
```

Code for the PI speed controller:

```
#define ENCODER_A 2
#define ENCODER_B 3
#define PWM_IN1 12
#define PWM_IN2 13
#define CPR 64

int32_t previous_position = 0;
int32_t position = 0;
```

```
int32_t previous_time = 0;
int32_t time = 0;

float velocity_previous = 0;
float velocity = 0;

void setup() {
  Serial.begin(9600);
  pinMode(ENCODER_A, INPUT);
  pinMode(ENCODER_B, INPUT);
  attachInterrupt(digitalPinToInterrupt(ENCODER_A), readEncoder,
RISING);
}

void loop() {
  int dutycycle = 10;
  int direction = 1;
  setMotor(direction, dutycycle, PWM_IN1, PWM_IN2);

  Serial.print("Velocity: ");
  Serial.println(velocity);
}

void setMotor(int8_t direction, uint8_t pwm_dutycycle, uint8_t
pin_in1, uint8_t pin_in2) {
  if (direction == 1) {
    digitalWrite(pin_in1, HIGH);
    analogWrite(pin_in2, 255 - pwm_dutycycle*255/100);
  } else if (direction == -1) {
    digitalWrite(pin_in2, HIGH);
    analogWrite(pin_in1, 255 - pwm_dutycycle*255/100);
  } else {
    digitalWrite(pin_in1, HIGH);
    digitalWrite(pin_in2, HIGH);
  }

}

float readEncoder() {
```

```
  uint8_t valueRead = digitalRead(ENCODER_B);

  if (valueRead) {
    position++;
  } else {
    position--;
  }

  int32_t time = micros();
  float dt = ((float) (time - previous_time))/1.0e6;
  previous_time = time;
  velocity_previous = velocity;
  velocity = valueRead/(dt*CPR);
}
```

Most

```
// Limit PID output if necessary for speed regulation?

/* Clock frequency */
#define CLK_FREQUENCY 16000000

/* Setup encoder pins for motor 1 */
#define ENCODER_A_1 2
#define ENCODER_B_1 22

/* Setup encoder pins for motor 2 */
#define ENCODER_A_2 3
#define ENCODER_B_2 24

/* Setup encoder pins for motor 3 */
#define ENCODER_A_3 18
#define ENCODER_B_3 26

/* Setup motor driver pins for motor 1 */
#define PWM_IN1_1 6
#define PWM_IN2_1 7

/* Setup motor driver pins for motor 2 */
#define PWM_IN1_2 8
```

```
#define PWM_IN2_2 44

/* Setup motor driver pins for motor 3 */
#define PWM_IN1_3 45
#define PWM_IN2_3 46

/* Motor limit switches */
#define LIMIT_SWITCH_1 19
#define LIMIT_SWITCH_2 20
#define LIMIT_SWITCH_3 21

#define CPR_1 10200
#define CPR_2 10200
#define CPR_3 10200

#define PID_SAMPLING_TIME 50

/* Struct definitions */
typedef struct {
  /* Controller gains */
  float Kp;
  float Ki;
  float Kd;

  /* Derivative  low-pass filter time constant */
  float tau;

  /* Output limits */
  float limMin;
  float limMax;

  /* Integrator limits */
  float limMinInt;
  float limMaxInt;

  /* Sample time (in seconds) */
  float T;

  /* Controller memory */
  float integrator;
```

```
  float prevError;
  float differentiator;
  float prevMeasurement;

  /* Controller output */
  float out;
} PIDController;

/* PID update time (change to dedicated timer later?) */
uint32_t time_to_execute = PID_SAMPLING_TIME;

// Hold these positions when time to update

/* Position hold variables */
int32_t previous_encoder_count_1 = 0;
int32_t previous_encoder_count_2 = 0;
int32_t previous_encoder_count_3 = 0;

int32_t encoder_count_1 = 0;
int32_t encoder_count_2 = 0;
int32_t encoder_count_3 = 0;

float angle_difference_1 = 0;
float angle_difference_2 = 0;
float angle_difference_3 = 0;

float initial_angle_1 = 0;
float initial_angle_2 = 50;
float initial_angle_3 = 210;

float angle_1 = 0;
float angle_2 = 0;
float angle_3 = 0;

float previous_angle_1 = 0;
float previous_angle_2 = 0;
float previous_angle_3 = 0;

// Velocity variables for each motor
float velocity_1 = 0;
```

```
float velocity_2 = 0;
float velocity_3 = 0;

// Setpoint variables for each motor
float setpoint_1 = initial_angle_1;
float setpoint_2 = initial_angle_2;
float setpoint_3 = initial_angle_3;

/* Motor 1 PID declaration */
PIDController pid_1 = { 0.7, 0.05, 0,
                        0,
                        -255, 255,
                        -127, 127,
                        PID_SAMPLING_TIME/1000.0,
                        0, 0, 0, 0,
                        0};

/* Motor 2 PID declaration */
PIDController pid_2 = { 0.25, 0.1, 0,
                        0,
                        -255, 255,
                        -127, 127,
                        PID_SAMPLING_TIME/1000.0,
                        0, 0, 0, 0,
                        0};

/* Motor 3 PID declaration */
PIDController pid_3 = { 0.2, 0.1, 0,
                        0,
                        -255, 255,
                        -127, 127,
                        PID_SAMPLING_TIME/1000.0,
                        0, 0, 0, 0,
                        0};

  /* Motor 1 PID declaration */
// PIDController pid_1 = { 5, 1, 0,
//                         0,
//                         -255, 255,
//                         -127, 127,
```

```
//                          PID_SAMPLING_TIME/1000.0,
//                          0, 0, 0, 0,
//                          0};

// /* Motor 2 PID declaration */
// PIDController pid_2 = { 0.1, 0.01, 0,
//                          0,
//                          -255, 255,
//                          -127, 127,
//                          PID_SAMPLING_TIME/1000.0,
//                          0, 0, 0, 0,
//                          0};

// /* Motor 3 PID declaration */
// PIDController pid_3 = { 0.1, 0.01, 0,
//                          0,
//                          -255, 255,
//                          -127, 127,
//                          PID_SAMPLING_TIME/1000.0,
//                          0, 0, 0, 0,
//                          0};

/* Max velocity profile */
// float max_velocity_profile[100] = {0};

/* PID functions */

void PIDController_Init(PIDController *pid) {
  /* Clear controller variables */
  pid->integrator = 0.0f;
  pid->prevError  = 0.0f;

  pid->differentiator  = 0.0f;
  pid->prevMeasurement = 0.0f;

  pid->out = 0.0f;
}

float PIDController_Update(PIDController *pid, float setpoint,
float measurement) {
```

```
  /*Error signal */
  float error = setpoint - measurement;

  /* Proportional */
  float proportional = pid->Kp * error;

  /* Integral */
  pid->integrator = pid->integrator + 0.5f * pid->Ki * pid->T *
(error + pid->prevError);

  /* Anti-wind-up via integrator clamping */
  if (pid->integrator > pid->limMaxInt) {
    pid->integrator = pid->limMaxInt;
  } else if (pid->integrator < pid->limMinInt) {
    pid->integrator = pid->limMinInt;
  }

  /* Derivative (band-limited differentiator) */
  pid->differentiator = -(2.0f * pid->Kd * (measurement - pid-
>prevMeasurement) /* Note: derivative on measurement, therefore
minus sign in front of equation! */
                        + (2.0f * pid->tau - pid->T) * pid-
>differentiator)
                        / (2.0f * pid->tau + pid->T);

  /* Compute output and apply limits */
  pid->out = proportional + pid->integrator + pid->differentiator;

  if (pid->out > pid->limMax) {
    pid->out = pid->limMax;
  } else if (pid->out < pid->limMin) {
    pid->out = pid->limMin;
  }

  /* Store error and measurement for later use */
  pid->prevError = error;
  pid->prevMeasurement = measurement;

  /* Return controller output */
  return pid->out;
```

```
}

/* Functions */

// Initial value and final value zero
void trapezoid(float *array, float slope, float max_value, float
sampling_time, float time) {
  uint16_t samples = time/sampling_time;
  uint16_t samples_slopes = max_value/(slope*sampling_time);
  for (uint16_t i = 0; i < samples_slopes; i++) {
    array[i] = slope*sampling_time*i;
  }

  for (uint16_t i = samples_slopes; i < samples + samples_slopes;
i++) {
    array[i] = max_value;
  }

  for (uint16_t i = samples + samples_slopes; i < samples +
samples_slopes*2; i++) {
    //array[i] = max_value + slope*sampling_time*(samples +
samples_slopes - 1 - i);
    array[i] = array[i-1] - slope*sampling_time;
  }

  for (uint16_t i = 0; i < samples + samples_slopes*2; i++) {
      Serial.print("Index: ");
      Serial.print(i);
      Serial.print(" Value: ");
      Serial.println(array[i]);
  }
}

void setMotor(int16_t pwm_dutycycle, uint8_t pin_in1, uint8_t
pin_in2) {
  if (pwm_dutycycle >= 0) {
    digitalWrite(pin_in1, HIGH);
    analogWrite(pin_in2, 255 - pwm_dutycycle);
  } else if (pwm_dutycycle < 0) {
    digitalWrite(pin_in2, HIGH);
```

```
    analogWrite(pin_in1, 255 + pwm_dutycycle);
  } else {
    digitalWrite(pin_in1, HIGH);
    digitalWrite(pin_in2, HIGH);
  }
}

/* Zeros motors to microswitch to develop reference */
void home(void) {
  // leaves interrupts disabled until home function finishes
executing
  // once home function finishes executing, reenable interrupts
and make sure to set all angles to initial angle
  // use setmotor to set pwm manually
  setMotor(75, PWM_IN1_1, PWM_IN2_1);
  setMotor(75, PWM_IN1_2, PWM_IN2_2);
  setMotor(75, PWM_IN1_3, PWM_IN2_3);

  while(!digitalRead(LIMIT_SWITCH_1) ||
!digitalRead(LIMIT_SWITCH_2) || !digitalRead(LIMIT_SWITCH_3)) {
    if (digitalRead(LIMIT_SWITCH_1)) {
      setMotor(0, PWM_IN1_1, PWM_IN2_1);
    }

    if (digitalRead(LIMIT_SWITCH_2)) {
      setMotor(0, PWM_IN1_2, PWM_IN2_2);
    }

    if (digitalRead(LIMIT_SWITCH_3)) {
      setMotor(0, PWM_IN1_3, PWM_IN2_3);
    }
  }
}

float motionProfile(float position, float final_position, float
velocity, float max_velocity, float acceleration, float
update_time, max_error) {
  if (position < final_position*(1 - max_error) || position >
final_position*(1 + max_error)) {
    if (position < final_position) {
```

```
    }

    if (fabs(position) < fabs((final_position -
initial_position)/2 + position)) {
      velocity = velocity + max_acceleration*update_time;
      if (velocity > max_velocity) {
        velocity = max_velocity;
      } else if (velocity < -max_velocity) {
        velocity = -max_velocity;
      }
    } else if (position )
  }

  if (velocity < -max_velocity) {
    velocity = -max_velocity;
  } else if (velocity > max_velocity) {
    velocity = max_velocity;
  } else {

  }

  return velocity;
}

/* Interrupt functions */
void readEncoder_1() {
  uint8_t valueRead = digitalRead(ENCODER_B_1);
  if (valueRead) {
    encoder_count_1++;
  } else {
    encoder_count_1--;
  }
}

void readEncoder_2() {
  uint8_t valueRead = digitalRead(ENCODER_B_2);
  if (valueRead) {
    encoder_count_2++;
```

```
  } else {
    encoder_count_2--;
  }
}

void readEncoder_3() {
  uint8_t valueRead = digitalRead(ENCODER_B_3);
  if (valueRead) {
    encoder_count_3++;
  } else {
    encoder_count_3--;
  }
}

uint8_t toggle = 0;
uint8_t pid_enable = 0;

ISR(TIMER1_COMPA_vect) {
    digitalWrite(13, toggle);
    toggle = ~toggle;

    /* Compute position as an angle */
    angle_1 = initial_angle_1 + encoder_count_1*360.0f/CPR_1;
    angle_2 = initial_angle_2 + encoder_count_2*360.0f/CPR_2;
    angle_3 = initial_angle_3 + encoder_count_3*360.0f/CPR_3;

    /* Calculate angular velocities of all motors */
    velocity_1 = (angle_1 -
previous_angle_1)/(PID_SAMPLING_TIME/1000.0f)*60.0f/360.0f;
    velocity_2 = (angle_2 -
previous_angle_2)/(PID_SAMPLING_TIME/1000.0f)*60.0f/360.0f;
    velocity_3 = (angle_3 -
previous_angle_3)/(PID_SAMPLING_TIME/1000.0f)*60.0f/360.0f;

    /* Hold the previous angular positions of all motors */
    previous_angle_1 = angle_1;
    previous_angle_2 = angle_2;
    previous_angle_3 = angle_3;

    /* Update all PIDs */
```

```
    PIDController_Update(&pid_1, setpoint_1, angle_1);
    PIDController_Update(&pid_2, setpoint_2, angle_2);
    PIDController_Update(&pid_3, setpoint_3, angle_3);

    setMotor(pid_1.out, PWM_IN1_1, PWM_IN2_1);
    setMotor(pid_2.out, PWM_IN1_2, PWM_IN2_2);
    setMotor(pid_3.out, PWM_IN1_3, PWM_IN2_3);
}

void setup() {
  Serial.begin(115200);

  /* Disables interrupts */
  SREG = 0x7F;

  /* Configure timer 1 for 50 ms interrupts */
  TCCR1A = 0;
  TCCR1B = 0;
  TCNT1 = 0;

  // Set compare match register for 50 Hz increments.
  OCR1A = 12499;
  // Turn on CTC mode.
  TCCR1B |= (1 << WGM12);
  // Set CS12, CS11 and CS10 bits for 1024 prescaler.
  TCCR1B |= (0 << CS12) | (1 << CS11) | (1 << CS10);
  // Disable timer compare interrupt.
  TIMSK1 |= (1 << OCIE1A);

  /* Configure PWM pins to be ~3.9 kHz */
  TCCR4B = TCCR4B & 0b11111010;
  TCCR5B = TCCR5B & 0b11111010;

  PIDController_Init(&pid_1);
  PIDController_Init(&pid_2);
  PIDController_Init(&pid_3);

  /* Setup pins for motor 1 */
  pinMode(ENCODER_A_1, INPUT);
  pinMode(ENCODER_B_1, INPUT);
```

```
  pinMode(LIMIT_SWITCH_1, INPUT);

  /* Setup pins for motor 2 */
  pinMode(ENCODER_A_2, INPUT);
  pinMode(ENCODER_B_2, INPUT);
  pinMode(LIMIT_SWITCH_2, INPUT);

  /* Setup pins for motor 3 */
  pinMode(ENCODER_A_3, INPUT);
  pinMode(ENCODER_B_3, INPUT);
  pinMode(LIMIT_SWITCH_3, INPUT);

  /* Setup ISR for motor 1 */
  attachInterrupt(digitalPinToInterrupt(ENCODER_A_1),
readEncoder_1, RISING);

  /* Setup ISR for motor 2 */
  attachInterrupt(digitalPinToInterrupt(ENCODER_A_2),
readEncoder_2, RISING);

  /* Setup ISR for motor 3 */
  attachInterrupt(digitalPinToInterrupt(ENCODER_A_3),
readEncoder_3, RISING);

  encoder_count_1 = 0;
  encoder_count_2 = 0;
  encoder_count_3 = 0;

  /* Homing function (leave before interrupt enable)*/

  /* Enables global interrupts */
  SREG = 0xFF;

  delay(5000);
}

void loop() {
  if (millis() < 10000) {
    setpoint_1 = 0;
    setpoint_2 = 0;
```

```
  setpoint_3 = 210;
}

if (millis() > 10000) {
  setpoint_1 = 100;
  setpoint_2 = 100;
  setpoint_3 = 180;
}

if (millis() > 15000) {
  setpoint_1 = 80;
  setpoint_2 = 80;
  setpoint_3 = 200;
}

if (millis() > 20000) {
  setpoint_1 = 360;
  setpoint_2 = 360;
  setpoint_3 = 190;
}

if (millis() > 25000) {
  setpoint_1 = 720;
  setpoint_2 = 720;
  setpoint_3 = 210;
}

if (millis() > 30000) {
  setpoint_1 = 540;
  setpoint_2 = 540;
  setpoint_3 = 540;
}

if (millis() > 35000) {
  setpoint_1 = 0;
  setpoint_2 = 0;
  setpoint_3 = 0;
}

delay(200);
```

```
  Serial.print("Time = ");
  Serial.print(millis());

  Serial.print(", Setpoint 1 = ");
  Serial.print(setpoint_1);
  Serial.print(", Setpoint 2 = ");
  Serial.print(setpoint_2);
  Serial.print(", Setpoint 3 = ");
  Serial.print(setpoint_3);
  Serial.println();

  Serial.print("Readings: Velocity 1 = ");
  Serial.print(velocity_1);
  Serial.print(" RPM, ");
  Serial.print(angle_1);
  Serial.print(" degrees, ");
  Serial.println();

  Serial.print("Readings: Velocity 2 = ");
  Serial.print(velocity_2);
  Serial.print(" RPM, ");
  Serial.print(angle_2);
  Serial.print(" degrees");
  Serial.println();

  Serial.print("Readings: Velocity 3 = ");
  Serial.print(velocity_3);
  Serial.print(" RPM, ");
  Serial.print(angle_3);
  Serial.print(" degrees, ");
  Serial.print(encoder_count_3);
  Serial.print(" click");
  Serial.println("\n");
}
```

Code for GUI:

```
using System;
using System.IO.Ports;
using System.Runtime.Serialization;
using System.Threading;
```

```
using System.Windows.Forms;

namespace _3031Project
{
    internal static class Program
    {
        /// <summary>
        ///  The main entry point for the application.
        /// </summary>
        ///



        [STAThread]
        static void Main()
        {



            // To customize application configuration such as set high DPI settings or
default font,
            // see https://aka.ms/applicationconfiguration.
            ApplicationConfiguration.Initialize();
            Application.Run(new Form1());
        }
    }
}
```

```
using System.Diagnostics;
using System.IO.Ports;

namespace _3031Project
{
    public partial class Form1 : Form
    {



        private SerialPort _serialPort;

        public Form1()
        {
            InitializeComponent();

            string[] ports = SerialPort.GetPortNames();
            comboBox1.DataSource = ports;

            this.KeyPreview = true;
```

```
            this.KeyDown += Form1_KeyDown;



        }

        private void SendCommand(string command)
        {
            if (_serialPort != null && _serialPort.IsOpen)
            {
                _serialPort.WriteLine(command);
                Debug.WriteLine("command sent");
            }
        }

        private void startbtn_Click(object sender, EventArgs e)
        {



            string currentPort = Convert.ToString(comboBox1.SelectedItem);
            MessageBox.Show(currentPort);
            _serialPort = new SerialPort(currentPort, 115200, Parity.None, 8,
StopBits.One);

            _serialPort.DataReceived += sp_DataRx;
            //_serialPort.Open();
            try
            {
                _serialPort.Open();
            }
            catch (IOException)
            {
                MessageBox.Show("Serial Port Timed Out");
            }

        }

        private void stopbtn_Click(object sender, EventArgs e)
        {
            if (_serialPort != null && _serialPort.IsOpen)
            {
                _serialPort.Close();
            }
        }

        private void sp_DataRx(object sender, SerialDataReceivedEventArgs e)
        {
```

```
    string data = _serialPort.ReadLine();
    //MessageBox.Show(data);

    // Trim whitespace and check if data is valid
    data = data.Trim();
    if (string.IsNullOrEmpty(data))
        return;

    // in the video, he used ReadLine in VS, and Serial.println in arduino

    m1vbx.Invoke(new Action(() =>
    {
        string[] subs = data.Split(':');

        switch (subs[0])
        {
            case "m1p":
                m1pbx.Text = subs[1] + " deg";
                break;
            case "m1v":
                m1vbx.Text = subs[1] + " RPM";
                break;
            case "m2p":
                m2pbx.Text = subs[1] + " deg";
                break;
            case "m2v":
                m2vbx.Text = subs[1] + " RPM";
                break;
            case "m3p":
                m3pbx.Text = subs[1] + " deg";
                break;
            case "m3v":
                m3vbx.Text = subs[1] + " RPM";
                break;
            default:
                MessageBox.Show("Invalid Data Recived" + data);
                break;
        }
    }));
}

private void Form1_KeyDown(object sender, KeyEventArgs e)
{
    switch (e.KeyCode)
    {
        case Keys.Up:
            SendCommand("grs:inc");
            break;
        case Keys.Down:
            SendCommand("grs:dec");
            break;
        case Keys.Left:
```

```
            SendCommand("gts:inc");
            break;
        case Keys.Right:
            SendCommand("gts:dec");
            break;
        case Keys.PageUp:
            SendCommand("ghs:inc");
            break;
        case Keys.PageDown:
            SendCommand("ghs:dec");
            break;

    }

}
private void Form1_FormClosing(object sender, FormClosingEventArgs e)
{
    if (_serialPort != null && _serialPort.IsOpen)
    {
        _serialPort.Close();
    }

}

private void diagBtn_Click(object sender, EventArgs e)
{
    SendCommand("cmd:diag");
    //use serial.readstring on the arduino
}

private void m1fBtn_Click(object sender, EventArgs e)
{
    SendCommand("m1s:fwd");
}

private void m1rBtn_Click(object sender, EventArgs e)
{
    SendCommand("m1s:rev");
}

private void m2fBtn_Click(object sender, EventArgs e)
{
    SendCommand("m2s:fwd");
}

private void m2rBtn_Click(object sender, EventArgs e)
{
    SendCommand("m2s:rev");
}
```

```
        private void m3fBtn_Click(object sender, EventArgs e)
        {
            SendCommand("m3s:fwd");
        }

        private void m3vBtn_Click(object sender, EventArgs e)
        {
            SendCommand("m3s:rev");
        }

        private void obstBtn_Click(object sender, EventArgs e)
        {
            SendCommand("cmd:obst");
        }

        private void homeBtn_Click(object sender, EventArgs e)
        {
            SendCommand("cmd:home");
        }

        private void dp1Bx_DoubleClick(object sender, EventArgs e)
        {

            SendCommand("m1a:" + dp1Bx.Text);
        }

        private void dp2Bx_DoubleClick(object sender, EventArgs e)
        {
            SendCommand("m2a:" + dp2Bx.Text);
        }

        private void dp3Bx_DoubleClick(object sender, EventArgs e)
        {
            SendCommand("m3a:" + dp3Bx.Text);
        }
    }
}

namespace _3031Project
{
    partial class Form1
    {
        /// <summary>
        ///  Required designer variable.
        /// </summary>
        private System.ComponentModel.IContainer components = null;
```

```
        /// <summary>
        ///  Clean up any resources being used.
        /// </summary>
        /// <param name="disposing">true if managed resources should be disposed;
otherwise, false.</param>
        protected override void Dispose(bool disposing)
        {
            if (disposing && (components != null))
            {
                components.Dispose();
            }
            base.Dispose(disposing);
        }


        #region Windows Form Designer generated code


        /// <summary>
        ///  Required method for Designer support - do not modify
        ///  the contents of this method with the code editor.
        /// </summary>
        private void InitializeComponent()
        {
            startbtn = new Button();
            stopbtn = new Button();
            comboBox1 = new ComboBox();
            motor1lbl = new Label();
            motor2lbl = new Label();
            motor3lbl = new Label();
            gripper1lbl = new Label();
            m1vbx = new TextBox();
            m1pbx = new TextBox();
```

```
m2pbx = new TextBox();
m2vbx = new TextBox();
label1 = new Label();
label2 = new Label();
m3pbx = new TextBox();
m3vbx = new TextBox();
m1fBtn = new Button();
m1rBtn = new Button();
m2fBtn = new Button();
m2rBtn = new Button();
m3fBtn = new Button();
m3vBtn = new Button();
gfBtn = new Button();
glBtn = new Button();
grBtn = new Button();
gbBtn = new Button();
guBtn = new Button();
gdBtn = new Button();
diagBtn = new Button();
obstBtn = new Button();
homeBtn = new Button();
dp1Bx = new TextBox();
dp2Bx = new TextBox();
dp3Bx = new TextBox();
label3 = new Label();
SuspendLayout();
//
// startbtn
//
startbtn.Location = new Point(12, 12);
startbtn.Name = "startbtn";
startbtn.Size = new Size(112, 34);
```

```
startbtn.TabIndex = 0;
startbtn.Text = "Initialize";
startbtn.UseVisualStyleBackColor = true;
startbtn.Click += startbtn_Click;
//
// stopbtn
//
stopbtn.Location = new Point(130, 12);
stopbtn.Name = "stopbtn";
stopbtn.Size = new Size(112, 34);
stopbtn.TabIndex = 1;
stopbtn.Text = "Abort";
stopbtn.UseVisualStyleBackColor = true;
stopbtn.Click += stopbtn_Click;
//
// comboBox1
//
comboBox1.DropDownStyle = ComboBoxStyle.DropDownList;
comboBox1.FormattingEnabled = true;
comboBox1.Location = new Point(248, 12);
comboBox1.Name = "comboBox1";
comboBox1.Size = new Size(182, 33);
comboBox1.TabIndex = 2;
//
// motor1lbl
//
motor1lbl.AutoSize = true;
motor1lbl.Location = new Point(130, 72);
motor1lbl.Name = "motor1lbl";
motor1lbl.Size = new Size(77, 25);
motor1lbl.TabIndex = 3;
motor1lbl.Text = "Motor 1";
```

```
//
// motor2lbl
//
motor2lbl.AutoSize = true;
motor2lbl.Location = new Point(304, 72);
motor2lbl.Name = "motor2lbl";
motor2lbl.Size = new Size(77, 25);
motor2lbl.TabIndex = 4;
motor2lbl.Text = "Motor 2";
//
// motor3lbl
//
motor3lbl.AutoSize = true;
motor3lbl.Location = new Point(476, 72);
motor3lbl.Name = "motor3lbl";
motor3lbl.Size = new Size(77, 25);
motor3lbl.TabIndex = 5;
motor3lbl.Text = "Motor 3";
//
// gripper1lbl
//
gripper1lbl.AutoSize = true;
gripper1lbl.Location = new Point(717, 72);
gripper1lbl.Name = "gripper1lbl";
gripper1lbl.Size = new Size(71, 25);
gripper1lbl.TabIndex = 6;
gripper1lbl.Text = "Gripper";
//
// m1vbx
//
m1vbx.Location = new Point(92, 184);
m1vbx.Name = "m1vbx";
```

```
m1vbx.ReadOnly = true;
m1vbx.Size = new Size(150, 31);
m1vbx.TabIndex = 7;
//
// m1pbx
//
m1pbx.Location = new Point(92, 111);
m1pbx.Name = "m1pbx";
m1pbx.ReadOnly = true;
m1pbx.Size = new Size(150, 31);
m1pbx.TabIndex = 8;
//
// m2pbx
//
m2pbx.Location = new Point(268, 111);
m2pbx.Name = "m2pbx";
m2pbx.ReadOnly = true;
m2pbx.Size = new Size(150, 31);
m2pbx.TabIndex = 9;
//
// m2vbx
//
m2vbx.Location = new Point(268, 184);
m2vbx.Name = "m2vbx";
m2vbx.ReadOnly = true;
m2vbx.Size = new Size(150, 31);
m2vbx.TabIndex = 10;
//
// label1
//
label1.AutoSize = true;
label1.Location = new Point(12, 111);
```

```
label1.Name = "label1";
label1.Size = new Size(75, 25);
label1.TabIndex = 11;
label1.Text = "Position";
//
// label2
//
label2.AutoSize = true;
label2.Location = new Point(12, 190);
label2.Name = "label2";
label2.Size = new Size(62, 25);
label2.TabIndex = 12;
label2.Text = "Speed";
//
// m3pbx
//
m3pbx.Location = new Point(442, 111);
m3pbx.Name = "m3pbx";
m3pbx.ReadOnly = true;
m3pbx.Size = new Size(150, 31);
m3pbx.TabIndex = 16;
//
// m3vbx
//
m3vbx.Location = new Point(442, 184);
m3vbx.Name = "m3vbx";
m3vbx.ReadOnly = true;
m3vbx.Size = new Size(150, 31);
m3vbx.TabIndex = 17;
//
// m1fBtn
//
```

```
m1fBtn.Location = new Point(112, 241);
m1fBtn.Name = "m1fBtn";
m1fBtn.Size = new Size(112, 34);
m1fBtn.TabIndex = 18;
m1fBtn.Text = "Forward";
m1fBtn.UseVisualStyleBackColor = true;
m1fBtn.Click += m1fBtn_Click;
//
// m1rBtn
//
m1rBtn.Location = new Point(112, 281);
m1rBtn.Name = "m1rBtn";
m1rBtn.Size = new Size(112, 34);
m1rBtn.TabIndex = 19;
m1rBtn.Text = "Reverse";
m1rBtn.UseVisualStyleBackColor = true;
m1rBtn.Click += m1rBtn_Click;
//
// m2fBtn
//
m2fBtn.Location = new Point(286, 241);
m2fBtn.Name = "m2fBtn";
m2fBtn.Size = new Size(112, 34);
m2fBtn.TabIndex = 20;
m2fBtn.Text = "Forward";
m2fBtn.UseVisualStyleBackColor = true;
m2fBtn.Click += m2fBtn_Click;
//
// m2rBtn
//
m2rBtn.Location = new Point(286, 281);
m2rBtn.Name = "m2rBtn";
```

```
m2rBtn.Size = new Size(112, 34);
m2rBtn.TabIndex = 21;
m2rBtn.Text = "Reverse";
m2rBtn.UseVisualStyleBackColor = true;
m2rBtn.Click += m2rBtn_Click;
//
// m3fBtn
//
m3fBtn.Location = new Point(458, 241);
m3fBtn.Name = "m3fBtn";
m3fBtn.Size = new Size(112, 34);
m3fBtn.TabIndex = 22;
m3fBtn.Text = "Forward";
m3fBtn.UseVisualStyleBackColor = true;
m3fBtn.Click += m3fBtn_Click;
//
// m3vBtn
//
m3vBtn.Location = new Point(458, 281);
m3vBtn.Name = "m3vBtn";
m3vBtn.Size = new Size(112, 34);
m3vBtn.TabIndex = 23;
m3vBtn.Text = "Reverse";
m3vBtn.UseVisualStyleBackColor = true;
m3vBtn.Click += m3vBtn_Click;
//
// gfBtn
//
gfBtn.Font = new Font("Segoe UI", 11F);
gfBtn.Location = new Point(714, 111);
gfBtn.Name = "gfBtn";
gfBtn.Size = new Size(74, 49);
```

```
gfBtn.TabIndex = 24;
gfBtn.Text = "↑";
gfBtn.UseVisualStyleBackColor = true;
//
// glBtn
//
glBtn.Font = new Font("Segoe UI", 11F);
glBtn.Location = new Point(635, 166);
glBtn.Name = "glBtn";
glBtn.Size = new Size(74, 49);
glBtn.TabIndex = 25;
glBtn.Text = "←";
glBtn.UseVisualStyleBackColor = true;
//
// grBtn
//
grBtn.Font = new Font("Segoe UI", 11F);
grBtn.Location = new Point(794, 166);
grBtn.Name = "grBtn";
grBtn.Size = new Size(74, 49);
grBtn.TabIndex = 26;
grBtn.Text = "→";
grBtn.UseVisualStyleBackColor = true;
//
// gbBtn
//
gbBtn.Font = new Font("Segoe UI", 11F);
gbBtn.Location = new Point(714, 166);
gbBtn.Name = "gbBtn";
gbBtn.Size = new Size(74, 49);
gbBtn.TabIndex = 27;
gbBtn.Text = "↓";
```

```
gbBtn.UseVisualStyleBackColor = true;
//
// guBtn
//
guBtn.Font = new Font("Segoe UI", 9F);
guBtn.Location = new Point(634, 111);
guBtn.Name = "guBtn";
guBtn.Size = new Size(74, 49);
guBtn.TabIndex = 28;
guBtn.Text = "Up";
guBtn.UseVisualStyleBackColor = true;
//
// gdBtn
//
gdBtn.Font = new Font("Segoe UI", 9F);
gdBtn.Location = new Point(794, 111);
gdBtn.Name = "gdBtn";
gdBtn.Size = new Size(74, 49);
gdBtn.TabIndex = 29;
gdBtn.Text = "Down";
gdBtn.UseVisualStyleBackColor = true;
//
// diagBtn
//
diagBtn.Location = new Point(621, 241);
diagBtn.Name = "diagBtn";
diagBtn.Size = new Size(152, 34);
diagBtn.TabIndex = 33;
diagBtn.Text = "Diagonal Line";
diagBtn.UseVisualStyleBackColor = true;
diagBtn.Click += diagBtn_Click;
//
```

```
// obstBtn
//
obstBtn.Location = new Point(621, 281);
obstBtn.Name = "obstBtn";
obstBtn.Size = new Size(152, 34);
obstBtn.TabIndex = 34;
obstBtn.Text = "Traverse Wall";
obstBtn.UseVisualStyleBackColor = true;
obstBtn.Click += obstBtn_Click;
//
// homeBtn
//
homeBtn.Location = new Point(779, 241);
homeBtn.Name = "homeBtn";
homeBtn.Size = new Size(89, 74);
homeBtn.TabIndex = 35;
homeBtn.Text = "Home";
homeBtn.UseVisualStyleBackColor = true;
homeBtn.Click += homeBtn_Click;
//
// dp1Bx
//
dp1Bx.Location = new Point(92, 148);
dp1Bx.Name = "dp1Bx";
dp1Bx.Size = new Size(150, 31);
dp1Bx.TabIndex = 36;
dp1Bx.DoubleClick += dp1Bx_DoubleClick;
//
// dp2Bx
//
dp2Bx.Location = new Point(268, 148);
dp2Bx.Name = "dp2Bx";
```

```
dp2Bx.Size = new Size(150, 31);
dp2Bx.TabIndex = 37;
dp2Bx.DoubleClick += dp2Bx_DoubleClick;
//
// dp3Bx
//
dp3Bx.Location = new Point(442, 147);
dp3Bx.Name = "dp3Bx";
dp3Bx.Size = new Size(150, 31);
dp3Bx.TabIndex = 38;
dp3Bx.DoubleClick += dp3Bx_DoubleClick;
//
// label3
//
label3.AutoSize = true;
label3.Location = new Point(12, 151);
label3.Name = "label3";
label3.Size = new Size(62, 25);
label3.TabIndex = 39;
label3.Text = "D. Pos";
//
// Form1
//
AutoScaleDimensions = new SizeF(10F, 25F);
AutoScaleMode = AutoScaleMode.Font;
ClientSize = new Size(888, 367);
Controls.Add(label3);
Controls.Add(dp3Bx);
Controls.Add(dp2Bx);
Controls.Add(dp1Bx);
Controls.Add(homeBtn);
Controls.Add(obstBtn);
```

```
Controls.Add(diagBtn);
Controls.Add(gdBtn);
Controls.Add(guBtn);
Controls.Add(gbBtn);
Controls.Add(grBtn);
Controls.Add(glBtn);
Controls.Add(gfBtn);
Controls.Add(m3vBtn);
Controls.Add(m3fBtn);
Controls.Add(m2rBtn);
Controls.Add(m2fBtn);
Controls.Add(m1rBtn);
Controls.Add(m1fBtn);
Controls.Add(m3vbx);
Controls.Add(m3pbx);
Controls.Add(label2);
Controls.Add(label1);
Controls.Add(m2vbx);
Controls.Add(m2pbx);
Controls.Add(m1pbx);
Controls.Add(m1vbx);
Controls.Add(gripper1lbl);
Controls.Add(motor3lbl);
Controls.Add(motor2lbl);
Controls.Add(motor1lbl);
Controls.Add(comboBox1);
Controls.Add(stopbtn);
Controls.Add(startbtn);
Name = "Form1";
Text = "SCORBOT Control";
ResumeLayout(false);
PerformLayout();
```

```
}

#endregion

private Button startbtn;
private Button stopbtn;
private ComboBox comboBox1;
private Label motor1lbl;
private Label motor2lbl;
private Label motor3lbl;
private Label gripper1lbl;
private TextBox m1vbx;
private TextBox m1pbx;
private TextBox m2pbx;
private TextBox m2vbx;
private TextBox m3pbx;
private Label label1;
private Label label2;
private TextBox dp1Bx;
private TextBox dp2Bx;
private TextBox dp3Bx;
private TextBox m3vbx;
private TextBox textBox5;
private Button m1fBtn;
private Button m1rBtn;
private Button m2fBtn;
private Button m2rBtn;
private Button m3fBtn;
private Button m3vBtn;
private Button gfBtn;
private Button glBtn;
private Button grBtn;
```

```
        private Button gbBtn;
        private Button guBtn;
        private Button gdBtn;
        private Button diagBtn;
        private Button obstBtn;
        private Button homeBtn;
        private Label label3;
    }
}
```